

webexone²⁵

Curriculum

WebexOne 2025 Technical Training Program provides in-person technical training classes and hands-on labs across the Collaboration portfolio. It's where you'll get hands-on training across the entire solution, explore the latest innovations in AI, connect with influential leaders, and tap into expert insights.

The event takes place September 28 – October 1 in beautiful San Diego, CA.

Exploring the Possibilities of Webex APIs – Lab Guide

About this lab	4
Getting started	5
Webex Credentials	5
Postman Credentials	5
Log into the Webex Client	6
Log into Postman	6
Log into Webex for Developers	6
Visual Studio Code	7
1 – Making Your First API Requests	8
Step 1.1: Quick Test: Webex Developer Portal "Try It"	9
Step 1.2: Explore the Webex Public Workspace in Postman	11
Step 1.3: Use the "Webex Messaging" Collection in Postman	12
Step 1.4: Make an API Request with cURL	14
2 – Creating an Integration	15
Step 2.1: Create Your Webex Integration on the Developer Portal	16
Step 2.2: Manual OAuth 2.0 Authorization Code Flow (Browser + API Call)	17
Step 2.3: Configure OAuth 2.0 in Postman for Your Integration	20
Step 2.4: Make API Requests with Your Integration Token	22
3 – Building a Bot	23
Step 3.1: Create a Bot	24
Step 3.2: Send a message to yourself	25
Step 3.3: Create a Room and add yourself	26
Step 3.4: Adaptive card	27
Step 3.5: webex_bot	28
Step 3.6: webex_bot – Create your own function	30
Step 3.7: webex_bot – Adaptive card processing	32
4 – Deploying a Meeting Scheduler Service App	33
Step 4.1: Create a Webex Service App	34
Step 4.2: Request Admin Authorization	36
Step 4.3: Retrieve the Access and the Refresh Token	37
Step 4.4: Configure the Sample Service App	38
Step 4.5: Run the Sample Service App.	39
Step 4.6: Verify the Scheduled Meeting in Webex.	39
5 – Use Cases	40
Exercise 1: Create a feedback bot for your organization	41
Exercise 2: End-User Phone Provisioning Assistant	45
6 – Troubleshooting	48
Exercise 1: Webex Status API	49
Exercise 2: Admin Audit Events	53
Exercise 3: Reports	57
Exercise 4: Events	62
Exercise 5: Calling Troubleshooting	64
Exercise 6: Meetings Troubleshooting	69

About this lab

This hands-on lab aims to show participants the possibilities of Webex APIs and how they can be used for automation. We'll focus on real use cases to showcase how these APIs can be integrated into organizations.

This session will enable you to:

- Understand the key concepts and capabilities of Webex APIs.
- Integrate Webex APIs with bots and service apps to enable user communication and interaction.
- Explore practical examples of how Webex APIs can be applied.
- Discover how Webex APIs can be used for troubleshooting.

Getting started

Welcome to the Webex API Requests Lab! In this session, you'll learn how to interact with Webex using its powerful APIs. This "Getting Started" section will guide you through setting up your environment, ensuring you have everything you need to make your first API calls. Please use the specific lab credentials provided to you for all logins. To successfully complete this lab, you will be working with:

- Webex Client
- Postman
- Webex for Developers
- Visual Studio Code

Webex Credentials

Password will be the same for all the users:

- dCloud2856!

And your username name will be formatted based on your Pod number:

User	Username
Pod X	podx@cb127.dc-02.com

Postman Credentials

Username	Email	Password
PodX	podX@webexone25devs.wbx.ai	WebexOne2025!

Once you have identified your credentials you can continue.

Log into the Webex Client

To begin, you'll log into your dedicated Webex lab account. This will allow you to see the results of your API calls in real-time and interact with your Webex environment.

1. **Open the Webex Client:** Launch the Webex Desktop App on your lab workstation, or navigate to web.webex.com in your browser.
2. **Enter Lab Credentials:** When prompted, enter the **Webex email address and password** provided to you by the lab instructors.
3. **Verify Login:** Once logged in, you should see your Webex home screen. Take a moment to familiarize yourself with the interface.

Create Your Personal Webex Space

We'll create a dedicated Webex space (known as a "room" in the API world) that you can use for testing your API requests. This ensures your API interactions don't clutter other shared spaces.

1. **Start a New Space:** In the Webex client, click the "+" icon (or "Create a space" button) to start a new space.
2. **Name Your Space:** Give your space a clear, unique name, such as:
 - [Your Name/ID] - API Lab Space (e.g., JaneDoe-API Lab Space)
 - This will help you easily identify it later.
3. **Create Space:** Click "Create" or "Done" to finalize the space creation.
4. **Confirm:** You should now see your newly created space in your Webex client's space list.

Log into Postman

Postman will be our primary tool for making API requests. You'll log into a Postman account that has access to the Webex Public Workspace.

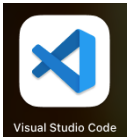
1. **Open Postman:** Launch the Postman Desktop App on your lab workstation
2. **Enter Lab Credentials:** When prompted, enter the **Postman email address and password** provided to you by the lab instructors.
3. **Verify Login:** Once logged in, you should see your Postman workspace. If you see a prompt to join a team, follow the lab instructor's guidance.

Log into Webex for Developers

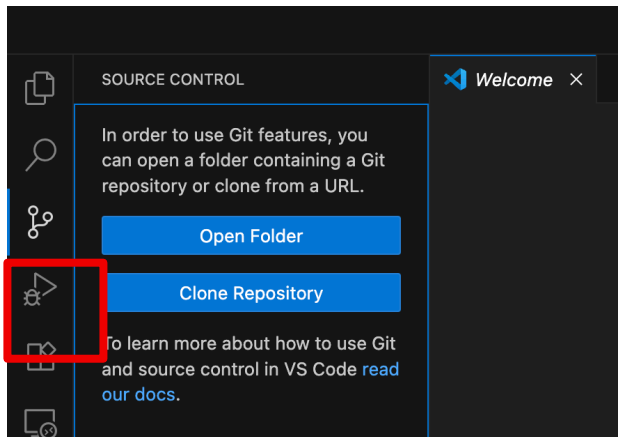
Use the same credentials to log into Webex for Developers at <https://developer.webex.com>

Visual Studio Code

Open Visual Studio Code from the desktop:

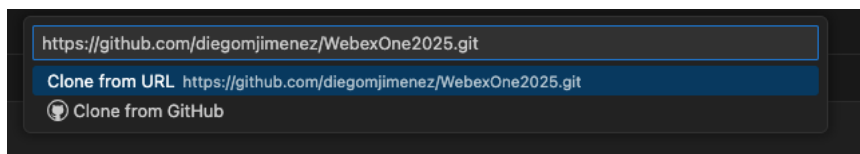


1. Go to the **Source Control** tab and click “Clone Repository”:



2. Type the following:

- `https://github.com/diegomjimenez/WebexOne2025.git`



3. Select a directory to save the project.
4. Click on “Yes, I trust the authors” if pop-up appears.
5. From the top bar, click on “...” > Terminal > New terminal:
6. Create a virtual environment:
 - `python -m venv webexone2025`
 - `.\webexone2025\Scripts\activate.ps1`
7. Install the requirements
 - `pip install -r requirements.txt`

1 – Making Your First API Requests

In this chapter, you'll dive into the exciting world of Webex APIs by making your very first requests. We'll explore different methods, starting with a quick test on the Webex Developer Portal, then moving to the powerful Postman environment, and finally, making a direct call using cURL.

Upon completion of this section, you will be able to:

1. Test Webex API requests on the Webex Developer Portal
2. Send Webex API requests using Postman
3. Execute Webex API calls with cURL from the command line

Reference:

<https://www.postman.com/webexdev/webex-public-workspace-beta>

Step 1.1: Quick Test: Webex Developer Portal "Try It"

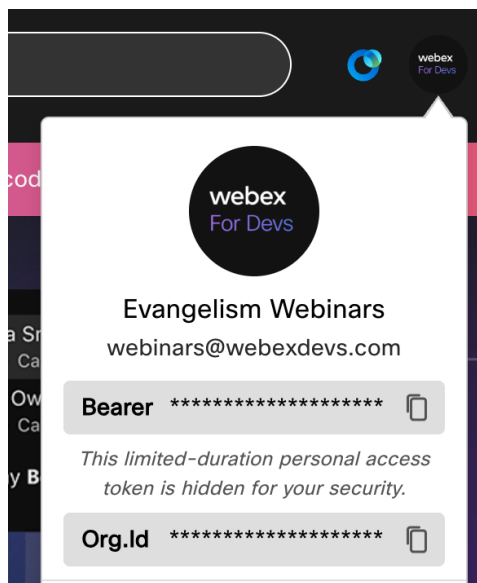
The Webex Developer Portal is an excellent resource for documentation and quick, one-off API tests. Let's get a temporary access token and make a simple call directly from the browser.

1. Open the Webex Developer Portal:

- Navigate to <https://developer.webex.com/> in your browser.

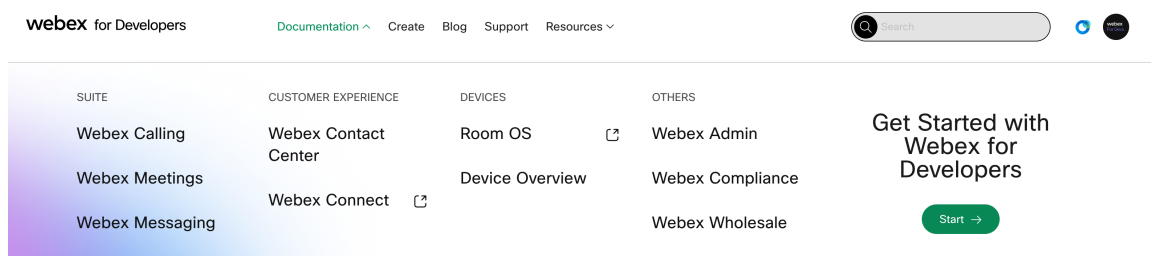
2. Get Your Temporary Access Token:

- On the developer portal homepage, locate the **"Profile"** section (usually at the top right).
- Ensure you are logged in with your **lab credentials**. If not, click "Login" and use the provided Webex email and password.
- Your temporary **Bearer Token** will be displayed. **Copy this token** to your clipboard. This token is valid for 12 hours.



3. Navigate to the Webex Messaging documentation area:

- In the header of the portal, click on the **"Documentation"** drop down to expand the "Mega Nav" bar.
- Under the **"SUITE"** section, select **"Webex Messaging"**.



4. Retrieve Your Own User Details (GET /people/me):

- In the sidebar navigation, under "**API Reference**", expand the "**All APIs**" section.
- Then expand "**People**" from the left-hand menu.
- Scroll down to find the "**Get My Own Details**" GET endpoint and select it.
- On the right side of the page, in the "**Try It**" section, paste your copied **Bearer Token** into the "Authorization" field (if it's not already populated with the "**Use personal access token**" toggle).
- Click the "**Run**" button.
- You should see a 200 OK status and a JSON response containing your Webex user details.
- Congratulations! You just made your first API request!

5. Find Your Lab Space ID (GET /rooms):

- In the left-hand menu, expand "**Rooms**".
- Scroll down to find the "**List Rooms**" GET endpoint and select it.
- Click the "**Run**" button.
- You'll get a 200 OK response with a list of your Webex spaces. **Carefully locate your personal lab space** (e.g., [Your Name/ID] - API Lab Space) and **copy its id value**. You'll need this in the next step!

6. Send a Message to Your Lab Space (POST /v1/messages):

- In the left-hand menu, select "**Messages**".
- Scroll down to find the "**Create a Message**" POST endpoint and select it.
- In the "Request Body" section, you'll see a JSON structure.
- Replace the placeholder for roomId with the id of your lab space that you just copied.
- Change the text field to a message like: "text": "Hello from the Webex Developer Portal 'Try It' tool!"
- Click the "**Run**" button.
- You should see a 200 OK response.

7. Verify Message in Webex Client:

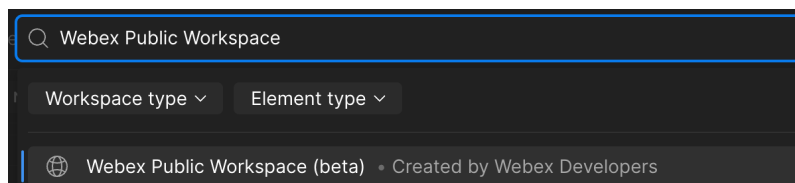
- Switch back to your Webex client. You should see the message you just sent appear in your personal lab space!
- *Note:* While great for quick tests, this "Try It" feature doesn't save your token or allow for complex workflows, which is why we'll use Postman next!

Step 1.2: Explore the Webex Public Workspace in Postman

Now, let's explore the Webex Public Workspace in Postman, which provides a much richer environment for API development.

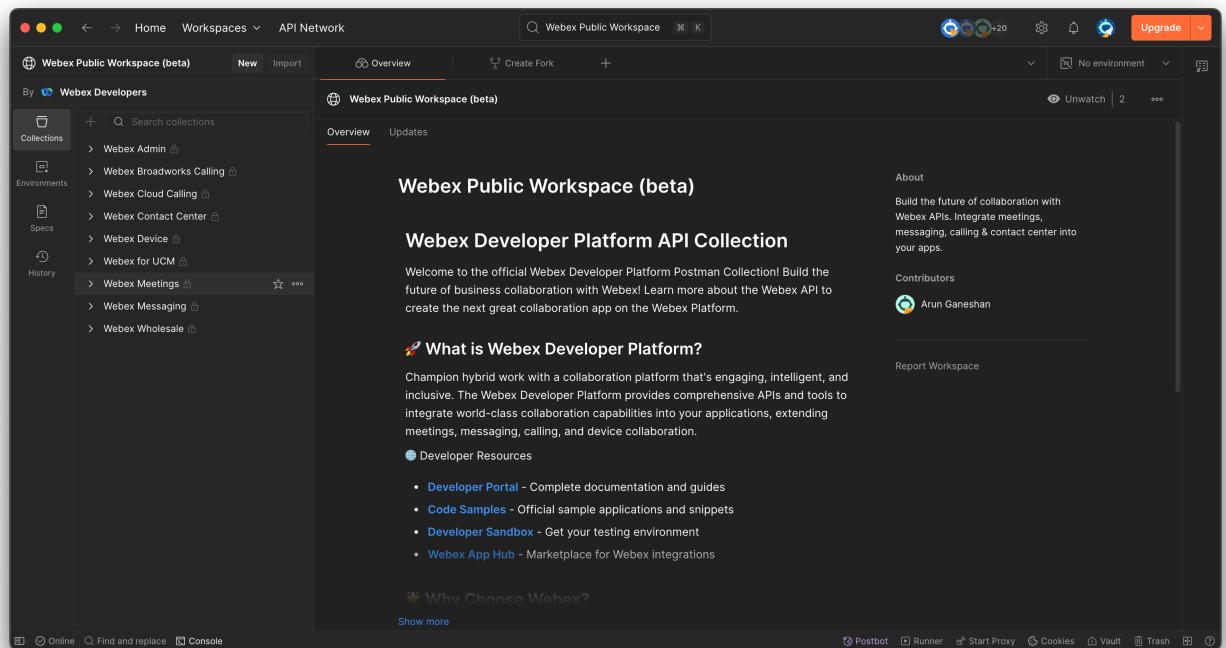
1. Open the Webex Public Workspace:

- In your Postman application (where you logged in with lab credentials), navigate to the Webex Public Workspace by searching for “**Webex Public Workspace**”



2. Observe the Workspace Structure:

- On the left sidebar, you'll see the “**Webex Public Workspace**” listed.
- Below it, notice the various **Collections** available (e.g., “Webex Messaging”, “Webex Meetings”, “Webex Calling”, etc.). These collections organize different sets of Webex APIs.
- *Remember:* A Postman Workspace can contain multiple Collections, Environments, and other elements.



Step 1.3: Use the "Webex Messaging" Collection in Postman

We'll now fork the "Webex Messaging" collection into your personal Postman space, set up your authentication, and make some practical API calls.

1. Fork the "Webex Messaging" Collection:

- In the Webex Public Workspace, locate the **"Webex Messaging"** collection in the left sidebar.
- Hover over the collection name and click the **"..." (ellipsis)** icon, then select **"Fork"**.
- In the dialog box, give the **Fork label** a name of **"Webex API Fork"**.
- Choose your **personal workspace** (it should be selected by default if you're in your own workspace).
- Also include the **"Webex Prod US1"** environment in the fork (we will use this environment later).
- Click **"Fork Collection"**.
- You should now see a copy of "Webex Messaging" in your personal workspace (usually listed under "Collections" in the left sidebar).

2. Configure Collection Authentication and Variable:

- Click on your forked **"Webex Messaging"** collection in the left sidebar.
- In the main Postman window, click the **"Variables"** tab.
- Add a new variable:
- **VARIABLE:** webex_token
- **VALUE:** Paste the **Bearer Token** you copied from the Webex Developer Portal in Step 1.1.2 here.
- Now, click the **"Authorization"** tab for the collection.
- For the **TYPE**, select **"Bearer Token"**.
- In the **Token** field, enter `{{webex_token}}`.
- *Explanation:* This sets up the collection to automatically use your `webex_token` variable for all requests within it!

3. Make Your First API Call with Postman (Get My Own Details):

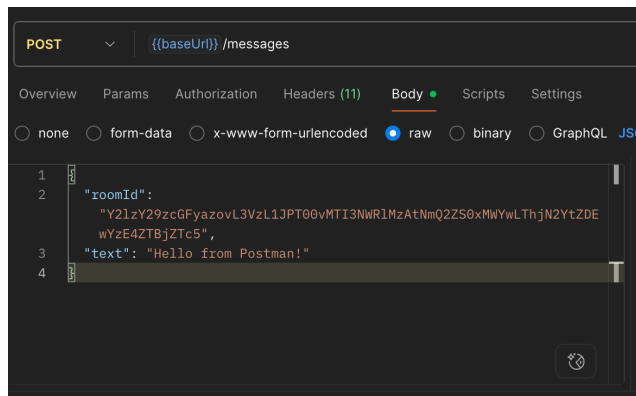
- In your forked "Webex Messaging" collection, expand the **"People"** folder.
- Click on the **GET Get My Own Details** request (which corresponds to GET /v1/people/me).
- In the main request window, ensure the "Authorization" tab shows "Inherit auth from parent" or "Bearer Token" and that `{{access_token}}` is in the Token field (if not inheriting).
- Click the blue **"Send"** button.
- You should receive a 200 OK response with your user details, just like in the Developer Portal.

4. List Your Webex Spaces (Rooms):

- In your forked "Webex Messaging" collection, expand the **"Rooms"** folder.
- Click on the **GET List Rooms** request (which corresponds to GET /v1/rooms).
- Uncheck all the **Query Params** in the **Params** tab.
- Click the blue **"Send"** button.
- You should receive a 200 OK response. In the JSON body, you'll see a list of your Webex spaces. **Carefully locate your personal lab space** (e.g., [Your Name/ID] - API Lab Space) and **copy its id value**. You'll need this in the next steps!

5. Send a Message to Your Lab Space (POST Create a Message):

- In your forked "Webex Messaging" collection, expand the "**Messages**" folder.
- Click on the **POST Create a Message** request (which corresponds to POST /v1/messages).
- In the main request window, click the "**Body**" tab.
- You'll see a JSON body. Remove all fields except for "**roomId**" and "**text**".
- Replace {{roomId}} with the actual id of your lab space (e.g., "roomId": "c2lkOi8v...").
- Change the text field to something personal, like "text": "Hello from Postman - this is much easier!".



- Click the blue "**Send**" button.
- You should receive a 200 OK response. Now, check your Webex Client – you should see the message appear in your lab space!

6. Read Messages in Your Lab Space (GET List Messages):

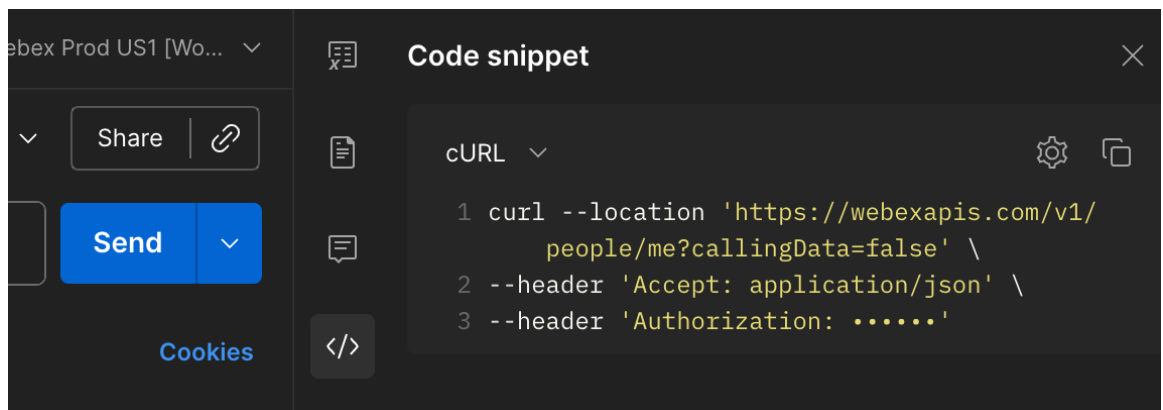
- Click on the **GET List Messages** request (which corresponds to GET /v1/messages).
- In the main request window, click the "**Params**" tab.
- Uncheck all **Query Params**.
- Check the **roomId** parameter, enter the id of your lab space that you copied earlier into the **Value**.
- Click the blue "**Send**" button.
- You should receive a 200 OK response with a list of messages in that space. You should see the message you just sent via Postman, and potentially the one you sent via the Developer Portal "Try It" tool!

Step 1.4: Make an API Request with cURL

Finally, let's see how to make an API call directly from your command line using cURL. Postman can even generate the cURL command for you!

1. Generate cURL from Postman:

- Go back to the **GET Get My Own Details** request you successfully ran in Postman (under "People API").
- On the right side of the request window, to the right of the "Send" button, click the "**Code**" link.
- A "Generate Code Snippets" window will appear. From the dropdown menu, select "**cURL**".
- **Copy the entire cURL command** that is displayed.



2. Open Your Terminal/Command Prompt:

- On your lab workstation, open a **Terminal** (macOS/Linux) or **Command Prompt/PowerShell** (Windows).

3. Execute the cURL Command:

- Paste the copied cURL command into your terminal/command prompt.
- **Important:** The cURL command generated by Postman will likely include your actual access_token value directly since it was set as the current value. If it still shows a placeholder, manually replace it with your **Bearer Token** that you copied in Step 1.1.2.
- *Example:* `curl --location --request GET 'https://api.webex.com/v1/people/me' \ --header 'Authorization: Bearer YOUR_ACTUAL_BEARER_TOKEN'`
- Press **Enter** to execute the command.

4. Observe the Output:

- The JSON response containing your user details will be printed directly in your terminal.

Congratulations! You've now made Webex API requests using the Developer Portal, Postman, and cURL. You're well on your way to becoming a Webex API pro!

2 – Creating an Integration

In Chapter 1, you used a temporary developer token for quick API tests. While convenient, this isn't how real-world applications securely access user data. This chapter introduces **Webex Integrations** and the **OAuth 2.0 Authorization Code Flow**, the standard for third-party applications to get secure, user-consented access to Webex APIs. You'll create your own integration, perform a manual OAuth flow, and then configure Postman to handle it seamlessly.

Upon completion of this section, you will be able to:

1. Create a Webex Integration
2. Perform a manual OAuth 2.0 authorization flow
3. Configure OAuth 2.0 authentication in Postman
4. Make API requests with the obtained token in Postman

Step 2.1: Create Your Webex Integration on the Developer Portal

First, you need to register your "application" (in this case, our Postman setup) with Webex. This process gives you a Client ID and Client Secret, which are essential for the OAuth flow.

1. **Navigate to "My Apps" on the Developer Portal:**
 - Open your browser and go to <https://developer.webex.com/my-apps>.
 - Ensure you are logged in with your **lab credentials**.
2. **Create a New Integration:**
 - Click the **"Create an Integration"** button.
3. **Fill in Integration Details:**
 - **Integration Name:** [Your Name/ID] - Lab Integration (e.g., JaneDoe-Lab Integration)
 - **Description:** Webex One Lab Integration for API testing.
 - **Icon:** Pick your favorite color icon.
 - **App Hub Description:** "Postman Integration for the Webex One 2025 Lab"
 - **Redirect URI(s):** This is critical! For Postman's built-in OAuth helper, we'll use a specific callback URL. Enter: <https://oauth.pstmn.io/v1/callback>
 - **Scopes:** These define what permissions your integration will request from the user. Check the following boxes:
 - spark:messages_write
 - spark:messages_read
 - spark:people_read
 - spark:rooms_read
 - Click **"Add Integration"**.
4. **Record Your Client ID and Client Secret:**
 - After creation, you'll see your integration's details. **Copy and save your Client ID and Client Secret and OAuth Authorization URL** to a temporary text file or notepad. These are sensitive credentials and will be needed shortly.
 - For best results, leave this page open for the rest of the lab.
 - *Note:* The Client Secret is only shown once. If you lose it, you'll have to regenerate it.

Step 2.2: Manual OAuth 2.0 Authorization Code Flow (Browser + API Call)

Now, let's manually walk through the steps a user and an application would take to get an access token using the Authorization Code flow. This will help you understand the underlying mechanics.

1. Use the Authorization URL:

- Copy the **“OAuth Authorization URL”** from the Integration Details page

OAuth Authorization URL

You can use the URL below to initiate an OAuth permission request for this app. It is configured with your redirect URI and app scopes. Be sure to update the state parameter.

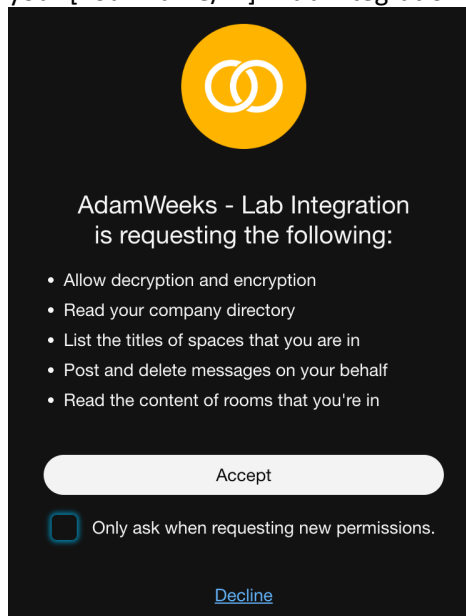
```
https://webexapis.com/v1/authorize?
client_id=C89a71f06fd1055f12d30dd70054558962ac711a01ca52fe77
c2bf9611672d6f1&response_type=code&redirect_uri=https%3A%2F%2F
Foauth.pstmn.io%2Fv1%2Fcallback&scope=spark%3Akms%20spark%3
Apeople_read%20spark%3Arooms_read%20spark%3Amessages_write
%20spark%3Amessages_read&state=set_state_here
```

Paste this URL into your browser's address bar:

- *Query Params Explanation:*
- `response_type=code`: We want an authorization code.
- `client_id`: Your integration's unique ID.
- `redirect_uri`: Where Webex sends the user back after authorization. Must match what you registered.
- `scope`: The permissions we're requesting.
- `state`: An optional parameter for security, usually a random string.?

2. Authorize the Integration (User Consent):

- Press Enter to navigate to the constructed URL.
- You'll be prompted to log in to Webex (if not already) and then asked to grant permission to your [Your Name/ID] - Lab Integration for the requested scopes.



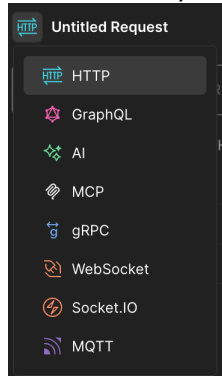
- Uncheck the **“Only ask when requesting new permissions.”** checkbox. This way, when we authorize again later, we will see the same dialog.
- Click **"Accept"**

3. Capture the Authorization Code:

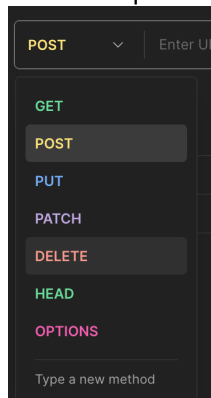
- Webex will redirect your browser to https://oauth.pstmn.io/v1/callback?code=YOUR_AUTHORIZATION_CODE&state=set_by_app.
- **Copy the authorization code value** from the URL in your browser's address bar. This is your temporary authorization code.

4. Exchange the Code for an Access Token (Postman API Call):

- Open Postman.
- Create a new HTTP Request (click the + tab).
- If it doesn't say "HTTP", you will need to change the type.



- Set the request method type to **POST**.



- Set the URL to: https://webexapis.com/v1/access_token
- Go to the **"Body"** tab, select **x-www-form-urlencoded**.
- Add the following key-value pairs:
 - Grant Type
 1. **KEY:** grant_type
 2. **VALUE:** authorization_code
 - Client ID
 1. **KEY:** client_id
 2. **VALUE:** YOUR_CLIENT_ID (Paste your Client ID)
 - Client Secret
 1. **KEY:** client_secret
 2. **VALUE:** YOUR_CLIENT_SECRET (Paste your Client Secret)
 - Auth Code
 1. **KEY:** code
 2. **VALUE:** YOUR_AUTHORIZATION_CODE (Paste the code you captured in step 3)
 - Redirect
 1. **KEY:** redirect_uri
 2. **VALUE:** <https://oauth.pstmn.io/v1/callback>
- Click **"Send"**.

5. **Observe the Token Response:**

- You should receive a 200 OK response containing your `access_token`, `refresh_token`, `expires_in` (how long the access token is valid), and `scope`.
- **Copy your new access_token.** This token was generated via your integration and user consent, making it a more secure way to access Webex APIs on behalf of a user.

6. **Update Postman Collection with New Token:**

- In your personal Postman workspace, navigate to your forked "**Webex Messaging**" collection.
- Click on the collection name in the left sidebar, then select the "**Variables**" tab in the main window.
- Find the `webex_token` variable you created in Chapter 1.
- Paste the `webex_token` you just copied from step 5 into the "**Value**" column for `webex_token`.

7. **Make an API Call with the Integration Token (List Rooms):**

- In your forked "Webex Messaging" collection, expand the "**Rooms**" folder.
- Click on the **GET List Rooms** request (which corresponds to `GET /v1/rooms`).
- Click the blue "**Send**" button.
- You should receive a 200 OK response. Verify that the rooms listed are correct for your lab account. This call was made using the `access_token` obtained through your integration!

Step 2.3: Configure OAuth 2.0 in Postman for Your Integration

Manually performing the OAuth flow is cumbersome. Postman has built-in support to automate much of this. Let's configure your forked "Webex Messaging" collection to use your new integration.

1. Open Your Forked "Webex Messaging" Collection:

- In your personal Postman workspace, navigate to your forked "Webex Messaging" collection.

2. Access Collection Authorization Settings:

- Click on your forked "Webex Messaging" collection in the left sidebar.
- In the main Postman window, click the "Authorization" tab.

3. Configure OAuth 2.0:

- For the **TYPE**, select "OAuth 2.0".
- Fill in the following details in the "GET NEW ACCESS TOKEN" window:
 - **Token Name:** Webex Lab Integration Token
 - **Grant Type:** Authorization Code
 - **Callback URL:** <https://oauth.pstmn.io/v1/callback> (This must match your registered Redirect URI)
 - **Auth URL:** <https://webexapis.com/v1/authorize>
 - **Access Token URL:** https://webexapis.com/v1/access_token
 - **Client ID:** YOUR_CLIENT_ID (Paste your Client ID from 2.1.4)
 - **Client Secret:** YOUR_CLIENT_SECRET (Paste your Client Secret from 2.1.4)
 - **Scope:** spark:people_read spark:rooms_read spark:messages_write spark:messages_read (Ensure these match what you registered, separated by spaces)
 - **Client Authentication:** Send client credentials in body
- Scroll down and click "Get New Access Token".
- Click "Proceed".

4. Authorize in Browser:

- Postman will open a browser window. You'll be prompted to log in to Webex (if necessary) and grant permission to your [Your Name/ID] - Lab Integration.
- Click "Accept" or "Authorize".

5. Postman Retrieves Token:

- The browser will redirect, and Postman will automatically capture the authorization code and exchange it for an access token.
- In the "MANAGE ACCESS TOKENS" window, you should see your Webex Lab Integration Token listed.
- Click "Use Token".

•

6. **Verify Collection Authorization:**

- Back in the collection's "Authorization" tab, the "Token" field should now show your Webex Lab Integration Token selected.
- Click **"Save"** for the collection.

Webex Messaging

Overview **Auth** ● Scripts Variables ● Runs

This authorization method will be used for every request in this collection. You can override this for individual requests.

Auth Type

OAuth 2.0

The authorization data will be automatically generated when you send the request. Learn more about [OAuth 2.0](#) authorization.

Add auth data to Request Headers

Current Token

This token is only available to you. Sync the token to let collaborators on this request use it.

Token Webex Lab Integration Token

.....

[Refresh](#)

Header Prefix ⓘ Bearer

Auto-refresh Token ☒

Your expired token will be auto-refreshed before sending a request.

(Your auth screen should look like this now)

Step 2.4: Make API Requests with Your Integration Token

Now that your Postman collection is configured to use the OAuth 2.0 flow, all requests within it will automatically use the access token obtained through your integration.

1. Retrieve Your Own User Details (GET /v1/people/me):

- In your forked “Webex Messaging” collection, expand the “**People API**” folder.
- Click on the **GET Get My Own Details** request.
- In the main request window, ensure the “Authorization” tab shows “Inherit auth from parent” or “Bearer Token” with the Webex Lab Integration Token selected.
- Click the blue “**Send**” button.
- You should receive a 200 OK response with your user details. This time, the token came from your integration!

2. Send a Message to Your Lab Space (POST Create a Message):

- In your forked “Webex Messaging” collection, expand the “**Messages API**” folder.
- Click on the **POST Create a Message** request.
- In the “Body” tab, replace {{roomId}} with the id of your lab space (you can get this from the GET List Rooms request if you don’t have it handy from Chapter 1).
- Change the text field to: “text”: “Hello from my Webex Integration via Postman!”.
- Click the blue “**Send**” button.
- Check your Webex client – the message should appear, sent using your integration’s permissions.

Congratulations! You've successfully created a Webex Integration, understood the OAuth 2.0 flow, and used it to securely make API requests. This is a fundamental concept for building robust Webex applications.

3 – Building a Bot

Upon completion of this section, you will be able to:

1. Create a **Bot** in Webex.
2. Send a message and find people using the Bot using Python.
3. Create a room and add a person using the Bot using Python.
4. Create and send an Adaptive Card.
5. Create an interactive bot using webex_bot python library.

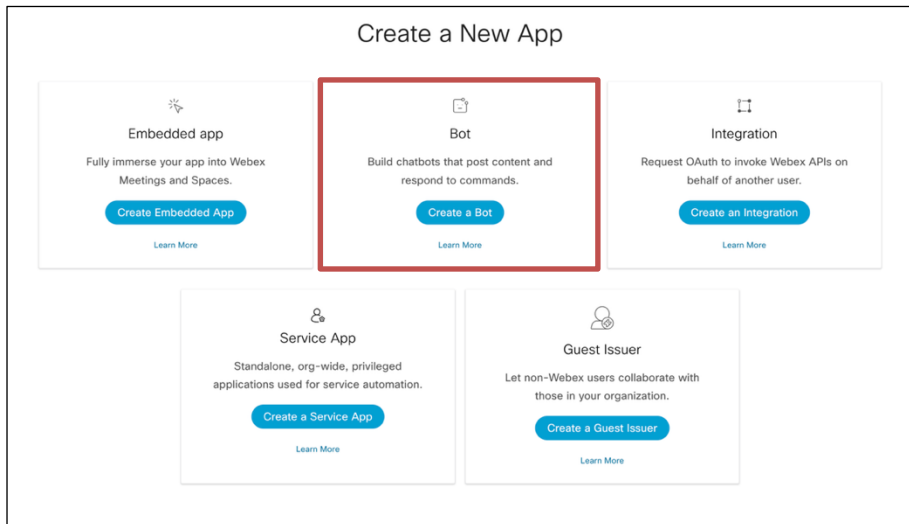
Reference:

<https://developer.webex.com/messaging/docs/bots>

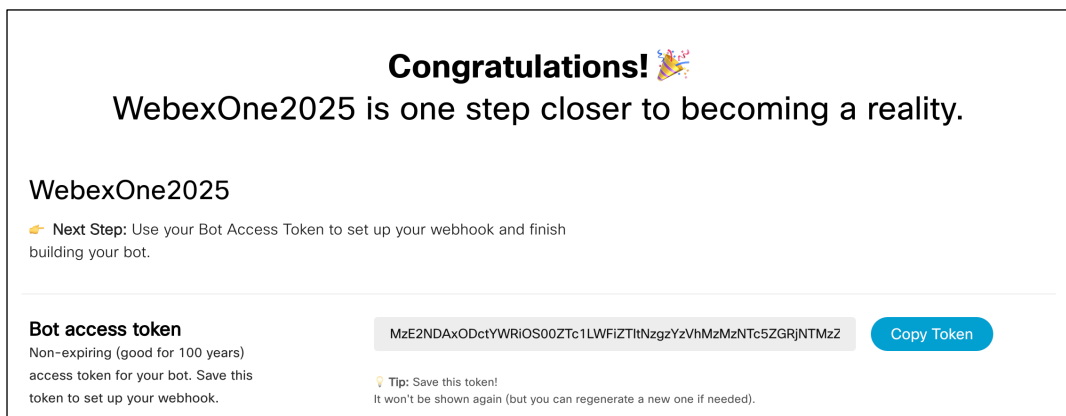
Step 3.1: Create a Bot

First you need to create your bot:

1. Log into developer.webex.com with credentials that were provided.
2. Up on the top right corner of the page, click your avatar and then select 'My Webex Apps'.
3. On the 'Create a New App' page, find the Bot card and click the 'Create a Bot' button.



4. Fill out the webform to register a new service app.
 - a. **Bot Name:** WebexOne-*USERNAME*
 - b. **Bot Username:** WebexOne-*USERNAME*
 - c. **Icon:** Select any color icon.
 - d. **Description:** "Bot for WebexOne"



▲ **Important:** Copy your **Bot access token** in your .env file as **BOT_TOKEN**.

Step 3.2: Send a message to yourself

In this step, you will send your first 1:1 message using the bot you just created. There are two options to achieve this:

1. In VS Code navigate to your .env file, and make sure to fill and save the following variables:
 - BOT_TOKEN
 - EMAIL
 - DOMAIN
2. Navigate to 03-bots/01_people.py and review the code. You will notice that there are two functions **find_people** and **all_people**.

⚠ **Warning:** **all_people** will not work if you are using a bot token as it requires admin privileges.

3. Make sure that in your terminal you are in the right folder:

- `cd 03-bots`

4. Run your code with the following command:

- `python 01_people.py`

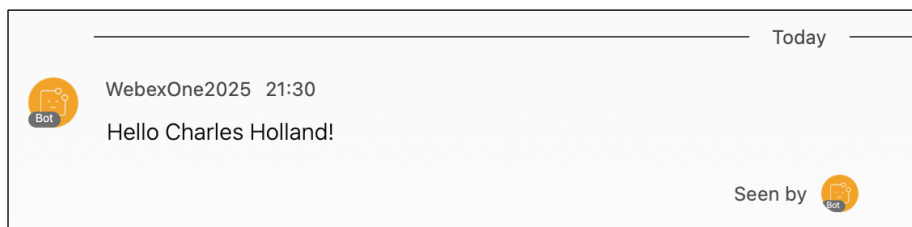
5. You should see the following in the console:

```
Name: Charles Holland, Email: ['cholland@cb136.dc-03.com']  
An error occurred: [400] Bad Request - Email, displayName, role, or id list should be specified.
```

6. Navigate to 03-bots/02_message.py and review the code. You will be using the function you created earlier to find yourself, so you can send a message to your own account.
7. Run your code with the following command:

- `python 02_message.py`

You should have received now the following message:



Step 3.3: Create a Room and add yourself

In this step, you will create a room with your bot and add your user to it.

1. Navigate to 03-bots/03_rooms.py and review the code.

There are two functions, one to create the room **create_webex_room** and another one to add a person to it **add_person_to_room**.

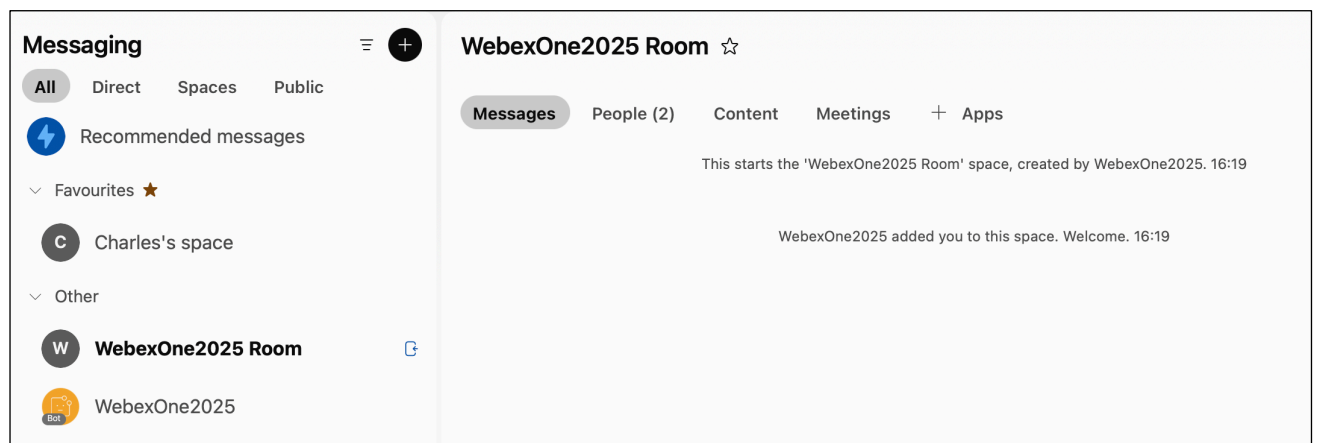
2. Run your code with the following command:

- `python 03_rooms.py`

3. You will see all the steps printed in the console:

```
Attempting to create room with title: 'WebexOne2025 Room'...
Room created successfully! Room ID: Y2lzY29zcGFyazovL3VybjpURUFN0nVzLXd1c3QzMl9yL1JPT00vMjcwMTF...
Attempting to find user with email: 'cholland@cb136.dc-03.com'...
Found user: Charles Holland, Person ID: Y2lzY29zcGFyazovL3VzL1BFT1BMRS83NmM00GUzNS02ZjE2LTQxZWJ...
Attempting to add Charles Holland to room ID: 'Y2lzY29zcGFyazovL3VybjpURUFN0nVzLXd1c3QzMl9yL1JF...
User 'Charles Holland' added to room successfully! Membership ID: Y2lzY29zcGFyazovL3VybjpURUFN0...
```

You should also see the newly created room appear in your Webex App:



Step 3.4: Adaptive card

In this step, you will explore how to create and send an Adaptive Card.

1. Navigate to 03-bots/04_adaptivecard and review the code.

You will notice that card content example, but ideally you will create your own card.

2. Navigate to the Buttons and Cards Designer:

- <https://developer.webex.com/buttons-and-cards-designer>

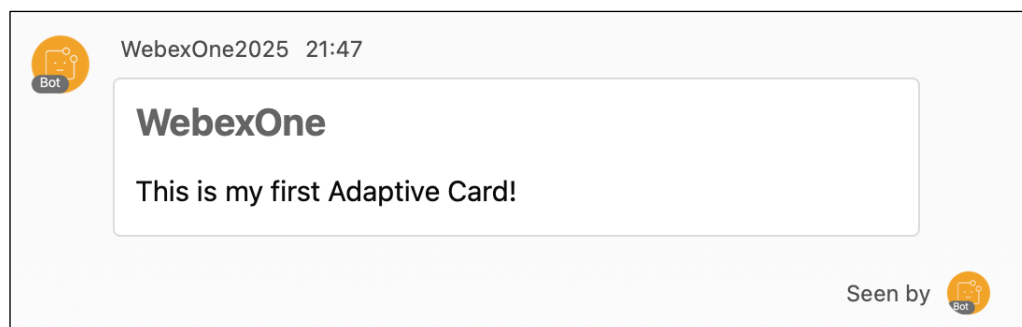
3. Use the UI to create your own card. Experiment with the available options to understand how they work. Once you're ready, your card's JSON code will appear below in the Card Payload Editor.

▲ Warning: When you copy the code from the Card Payload Editor, make sure to change **true** to **True** so it works correctly in Python.

4. Now, run your code with the following command:

- `python 04_adaptivecard.py`

If you have used the example card, you should receive the following:



Step 3.5: webex_bot

In this step and the following ones, you will work with the webex_bot library to get familiar with all the available options. You'll start with example cards and functions, and then learn how to add your own.

1. Navigate to 03-bots/05_webex_bot.py and review the code.

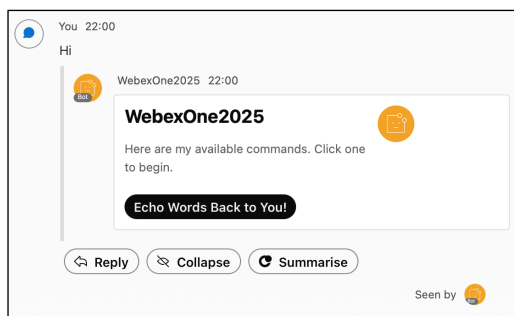
In this first example, you will use the default cards and functions provided to understand how the process works.

2. Execute the code with the following command and let it run:

- `python 05_webex_bot.py`

⚠ **Warning:** Wait until you see the message “WebSocket Opened.” appear in the console.

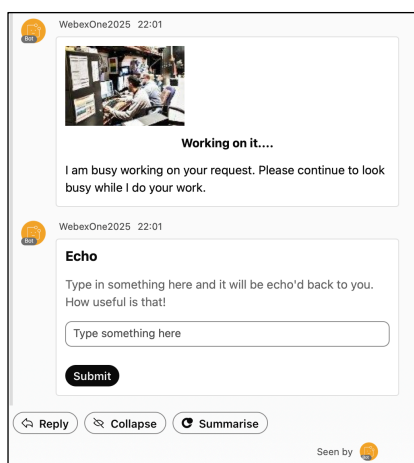
3. Send any message to your bot, and it will respond with the following card using the default function:



4. It is interesting to see how all the events are reflected in the console:

```
2025-09-14 22:00:29 [INFO] [webex_bot.webex_bot.webex_bot.process_incoming_message]:211 Message from cholland@cb136.dc-03.com: Webex Message:
{
  "id": "Y2lzY29zcGFyazovL3VybjpURUFN0NzLXdlc3QtM19yL01FU1NBR0UvNzVhZDJjMDAtOTFhNS0xMmYwLWI4MTctY2JlNTg2NmVmYTAY",
  "roomId": "Y2lzY29zcGFyazovL3VybjpURUFN0NzLXdlc3QtM19yL1JPT00vNThkZjg5MDAtOTBjZi0xMmYwLTK2MjAtMGJkOTU5YTNmNWU3",
  "roomType": "direct",
  "text": "Hi",
  "personId": "Y2lzY29zcGFyazovL3VzL1BFT1BMRS83NmM0OGUzNS02ZjE2LTQxZWItYjlmYy01ZGE5NTNmNWRlNTc",
  "personEmail": "cholland@cb136.dc-03.com",
  "created": "2025-09-14T20:00:27.840Z"
}
```

5. Once you click “Echo Words Back to You!”, an action will be executed and you will receive the following cards:



6. New event is also reflected in the console:

```
2025-09-14 22:01:02 [INFO] [webex_websocket_client.webex_bot.websockets.webex_websocket_client._process_incoming_websocket_message]:131 attachment_actions
se_64_id: Webex AttachmentAction:
{
  "id": "Y2lzY29zcGFyazovL3VybJpURUFN0NzLXdlc3QtMl9yL0FUVFEFDSE1FTlRfQUNUSU90L2g5ZmM0YzkwLTkxYTU0MTFmMC1iZThlWZk0TNkYwVlYtc4ZQ",
  "type": "submit",
  "messageId": "Y2lzY29zcGFyazovL3VybJpURUFN0NzLXdlc3QtMl9yL0F1FU1NBR0UvNzc5YTU2MDAtOTFhNS0xMmYwLWJjMzctNzliNTM4ZjZkxMMW4",
  "inputs": {
    "thread_parent_id": "75ad2c00-91a5-11f0-b817-cbe5866efa02",
    "command_keyword": "echo"
  },
  "personId": "Y2lzY29zcGFyazovL3VzL1BFT1BMRS83NmM0OGUzNS02ZjE2LTQxZWItYjlmYy01ZGE5NTNmNWRlNTc",
  "roomId": "Y2lzY29zcGFyazovL3VybJpURUFN0NzLXdlc3QtMl9yL1JPT00vNThkZjg5MDAtOTBjZi0xMmYwLWJjMzctMjAtMGJkOTU5YTNmNWU3",
  "created": "2025-09-14T20:01:01.913Z"
}
```

7. Type something in the box and click Submit. You should receive a new message, and the previous card will be deleted:



8. You should be able to see the message you sent in the console:

```
2025-09-14 22:01:44 [INFO] [webex_websocket_client.webex_bot.websockets.webex_websocket_client._process_incoming_websocket_message]:131 attachment_actions
se_64_id: Webex AttachmentAction:
{
  "id": "Y2lzY29zcGFyazovL3VybJpURUFN0NzLXdlc3QtMl9yL0FUVFEFDSE1FTlRfQUNUSU90L2EzM2FkYWwLTkxYTU0MTFmMC1iYzM3LTc5YjUzOGY5MTFjOA",
  "type": "submit",
  "messageId": "Y2lzY29zcGFyazovL3VybJpURUFN0NzLXdlc3QtMl9yL0F1FU1NBR0UvOGM3ZTYzOTAtOTFhNS0xMmYwLWJjMzctOGQ2MjQyOTRhY2My",
  "inputs": {
    "message_typed": "Testing echo!",
    "callback_keyword": "echo_callback"
  },
  "personId": "Y2lzY29zcGFyazovL3VzL1BFT1BMRS83NmM0OGUzNS02ZjE2LTQxZWItYjlmYy01ZGE5NTNmNWRlNTc",
  "roomId": "Y2lzY29zcGFyazovL3VybJpURUFN0NzLXdlc3QtMl9yL1JPT00vNThkZjg5MDAtOTBjZi0xMmYwLWJjMzctMjAtMGJkOTU5YTNmNWU3",
  "created": "2025-09-14T20:01:44.266Z"
}
```


Step 3.6: webex_bot – Create your own function

In this step, you will learn how to create your own function and add it to the bot.

1. Navigate to 03-bots/06_webex_bot-2.py and review the code.

You will be using WebexPythonSDK to send a message as shown previously.

▲ **Note:** Some changes have been made:

- **approved_domains** is used to restrict bot access to people inside your domain.
- **include_demo_commands** is set to False to hide the Echo function.
- **delete_previous_message** is set to True to delete the previous card and avoid duplicate actions.
- **bot_name** is the name that the bot will display on the cards.
- **command_keyword** is the text that you can use to invoke a function instead of the assistant.
- **help_message** will be the name of your function.

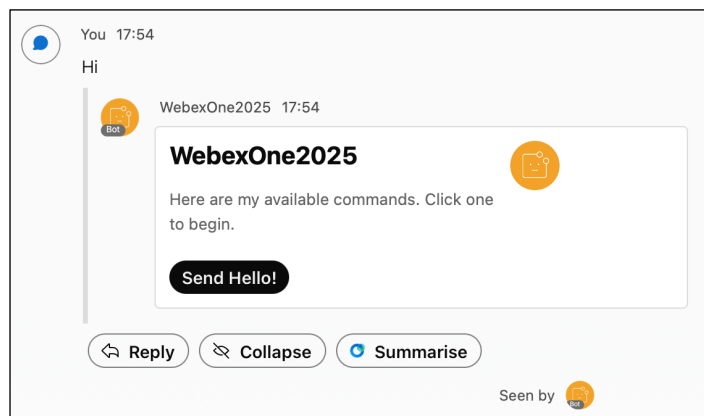
You can also access the information previously displayed in the console from the **attachment_actions** variable.

2. Execute the code with the following command and let it run:

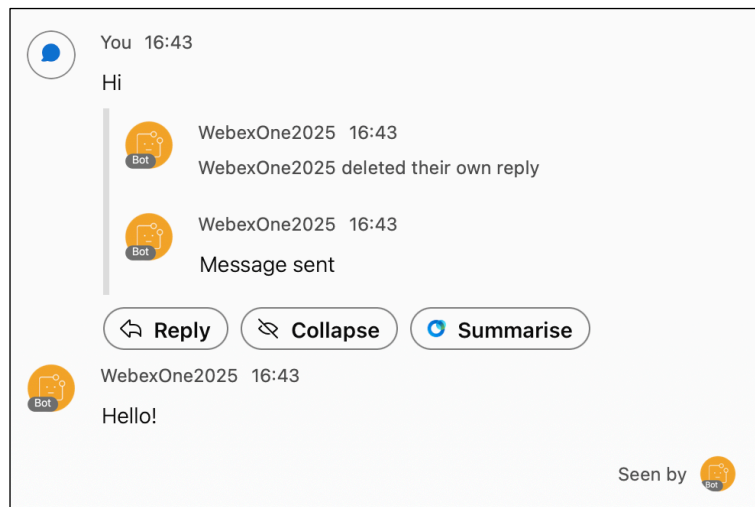
- `python 06_webex_bot-2.py`

▲ **Warning:** Wait until you see the message “WebSocket Opened.” appear in the console.

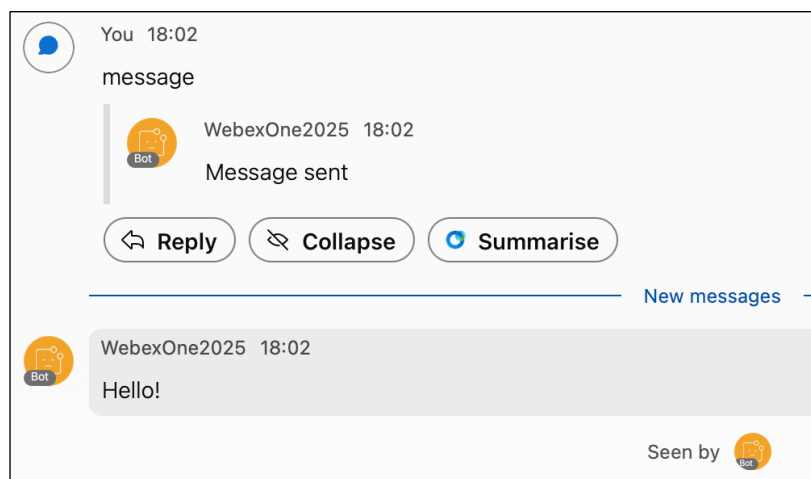
3. Send any message to your bot and you will get the following card with your new function **Send Hello!**



4. Click **Send Hello!**. The card will be deleted, you will receive a new message confirming that the message has been sent, and you should also see your **Hello!** message:



5. To directly invoke this function, you can text your bot with **message** keyword. This will automatically send you the **Hello!** message:



Step 3.7: webex_bot – Adaptive card processing

In this final step, you will send an Adaptive Card with multiple fields and use the submitted data to trigger an action.

1. Navigate to 03-bots/07_webex_bot-3.py and review the code.

▲ **Note:** Some changes have been added:

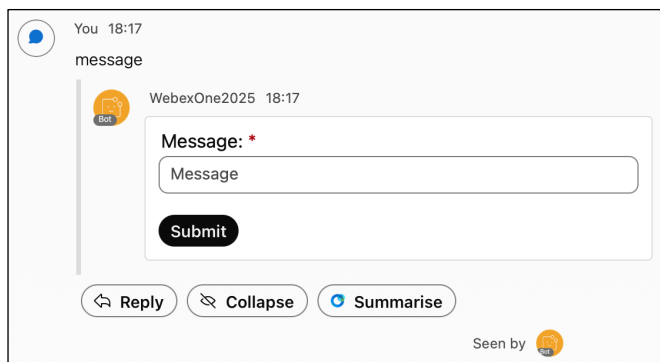
- **chained_commands** are the functions that will trigger this function when card data is submitted.
- **quote_info** formats your response.
- **Response** object allows you to send the Adaptive Card.

2. Execute the code with the following command and let it run:

- `python 07_webex_bot-3.py`

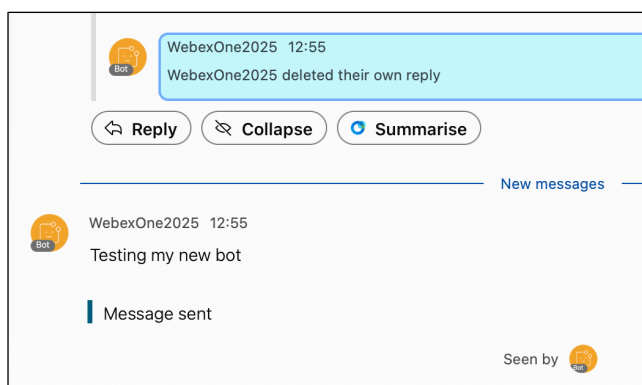
▲ **Warning:** Wait until you see the message “WebSocket Opened.” appear in the console.

3. Text **message** to your bot to invoke your function directly:



4. Enter a message and click Submit.

The previous card will be deleted. You should then receive both your message and a formatted notification confirming that your message has been sent:



4 – Deploying a Meeting Scheduler Service App

Upon completion of this section, you will be able to:

1. Create a **Service App** in Webex.
2. Request admin authorization to allow the Service App and observe the admin tasks in Control Hub.
3. Retrieve an access and refresh token to be used in a sample Service App.
4. Configure a sample Python-based Service App with the access and refresh token.
5. Run the sample Service App with admin permissions to schedule a meeting on behalf of a test user.
6. Verify the scheduled meeting in Webex.

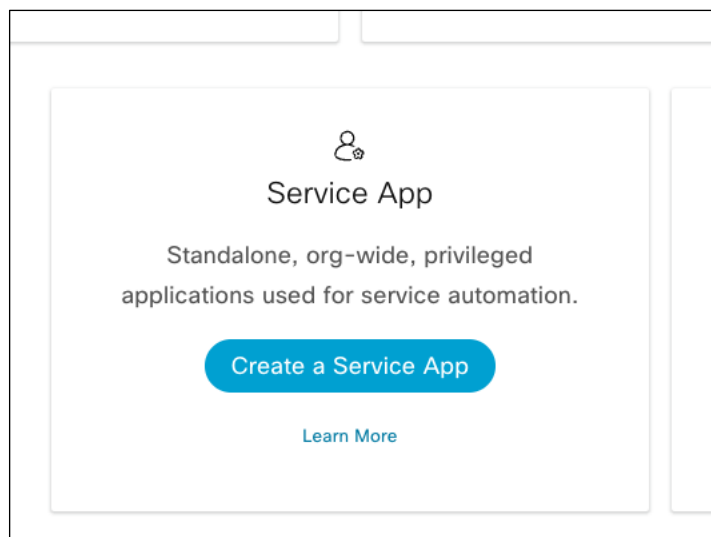
Reference:

<https://developer.webex.com/create/docs/service-apps>

Step 4.1: Create a Webex Service App

First is to register a Service App in the Webex Developer portal.

1. Log into developer.webex.com with credentials that were provided.
2. Up on the top right corner of the page, click your avatar and then select 'My Webex Apps'.
3. On the 'Create a New App' page, find the Service App card and click the 'Create a Service App' button.



4. Fill out the webform to register a new service app.
 - a. **App Name:** WebexOne-username
 - b. **Icon:** *Select any color icon.*
 - c. **Description:** "Test app only"
 - d. **Contact Email:** Use the email from the Webex login.

New Service App

App name*
Name of your service app as it will appear in Webex.

CL-user-42-test

Icon*
Upload your own or select from our defaults. Must be exactly 512x512px in JPEG or PNG format.


Edit

Description*
Provide some details about what your app does, how it benefits users, and most importantly, how a user can get started using it. The description should be under 1024 characters. You can use bullets, links, and Markdown formatting. If your app is listed on the Webex App Hub, this field will be used as the listing's description.



Test app only.

Supported markdown1010 characters remaining

Contact Email*

station42@webex-service-apps.wbx.ai

- e. **Select Scopes:** meeting:admin_schedule_write

Scopes
Scopes define the level of access that your service app requires.
[Learn more](#)

scroll through many options

☐ meeting:admin_participants_read
Read participant information from meetings for all Webex users of your organization

☒ meeting:admin_schedule_write
Create, manage, or cancel meetings of all Webex users of your organization

☐ meeting:admin_schedule_read
Retrieve meetings of all Webex users of your organization

By creating this app, you accept the [Terms of Service](#) and [Privacy Statement](#).

Cancel

Add Service App

5. Click the '**Add Service App**' button to finish registration.

Step 4.2: Request Admin Authorization

After successfully registering the Service App, you are taken to a page that contains a *Client ID* and *Client Secret*. This is also where you request admin authorization for the Service App.

1. Copy & paste the Client ID and Client Secret values into a notepad for later use. Do note, this client secret is only shown once.
2. Click the **'Request admin authorization'** button.

Congratulations!

Your app has been created.

CL-user-42-test

Admin Authorization

For use by other users inside your organization.

Request admin authorization

Let your administrator know you'd like your app approved for use inside your org.

Webex App Hub

Public catalog of apps available for all Webex users.

Submit to Webex App Hub

Tip: Submissions are optional.
Read our [Guidelines](#) to see if it makes sense to promote your app on Webex App Hub.

Authorization

Learn more about authentication in the [Apps & OAuth Guide](#).

Client ID

[REDACTED]

Copy

Client Secret

[REDACTED]

Copy

3. *The lab instructor will demonstrate the actions taken by the Webex administrator to authorize a Service App in Control Hub.*

Step 4.3: Retrieve the Access and the Refresh Token

After the admin authorizes your Service App registration, you can retrieve the access tokens.

1. Refresh the page that displays the *Client ID* and *Client Secret*.
2. In the '**Org Authorizations**' section, select the **PWB Webex Suite 5-2025** org from the dropdown menu.
3. Paste in the *Client Secret* value from your notepad in the field below, then click the 'Generate tokens' button.
4. Copy & paste the refresh_token and access_token values in your notepad for later use. *Both values are only shown once.*

Org Authorizations

Select an org from the dropdown menu or search by org ID. Enter your client secret to generate tokens.

PWB Webex Suite 5-2025 ▾

Org name	Org ID
PWB Webex Sui...	Y2IzY29zcGFyazovL3VzL09SR0FOSVpBVEI...

Generate tokens

refresh_token

.....

✓ Copied

access_token

.....

✓ Copied

Step 4.4: Configure the Sample Service App

Now that the access and refresh tokens are ready, we can insert them into the sample Python app and configure the meeting using a code editor.

A. Open the .env file in Visual Studio Code:

1. Go to **line 7** of the code and replace the text with the **clientID** value from your notepad.

- CLIENTID= "EXAMPLEC3ad18d5cb9bd01571e9b038438819c2b1"

2. On **line 8**, replace the text with the **secretID** value from your notepad.

- SECRETID= "EXAMPLEC3ad18d5cb9bdu05e9b0f130h88192b1"

3. On **line 9**, replace the text with the **access_token** value from your notepad.

- WEBEX_ACCESS_TOKEN= 'EXAMPLEQ2Y0_P03_0e01-g3-b55n-y06at4'

4. On **line 10**, replace the text with the **refresh_token** value from your notepad.

- REFRESH_TOKEN= 'EXAMPLE1hMt2_PA1_0e01-a56a-b0Gj204'

B. Open the 04-serviceapps/01_serviceapp.py file in Visual Studio Code:

1. Scroll to **line 69**, after 'title': replace the text and give your meeting a unique name.

- 'title': 'Automatic Meeting Example',

2. **Save** the 01_serviceapp.py file.

Step 4.5: Run the Sample Service App.

With all the Service App components added to Python code, the sample app is now ready to be initialized.

1. Go back to the command line window in the 04-serviceapps directory.
2. Launch the 01_serviceapp.py file from that location with this command:
 - `python3 01_serviceapp.py`
3. A success will return a statusCode 200 along with the the newly created meeting details in JSON format.
 - *statusCode: 200*

{'id': '7f36776aa9034d6091ef00dcee83ced2', 'meetingNumber': '26607444195', 'title': 'Important Customer Meeting', 'password'... .. }

Step 4.6: Verify the Scheduled Meeting in Webex.

You can quickly confirm that the Service App scheduled the meeting on behalf of your test user with one more API call.

1. Go back to the browser that is logged in as your test user on developer.webex.com.
2. Navigate to the List Meetings API reference page.
 - <https://developer.webex.com/docs/api/v1/meetings/list-meetings>
3. Make an API call to list meetings for the test user by adding your personal access token to the 'Authorization' header (after Bearer) and then clicking the blue 'Run' button (scroll down, along the right side).
4. The response should return a 200/OK and show the meeting information that was scheduled by the Service App on behalf of the test user

5 – Use Cases

In this section, you will put your Webex API knowledge into practice by building real-world solutions. You will explore how bots and integrations can streamline communication and automate device provisioning in an organization.

Upon completion of this section, you will be able to:

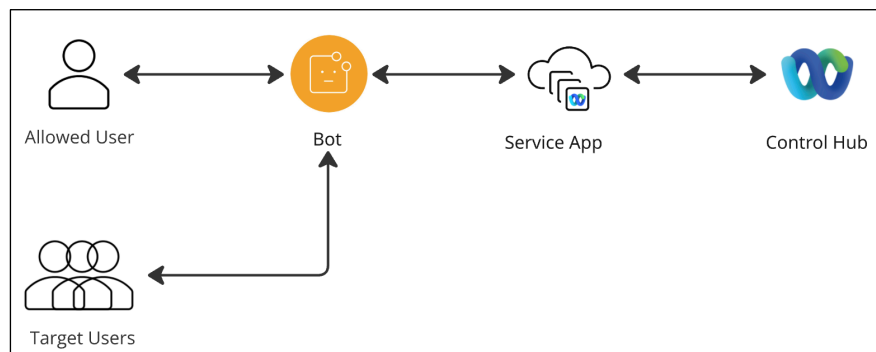
1. Apply the concepts that you have learned to create practical solutions using Webex APIs.
2. Build a bot that enables permitted users to collect feedback from all users in the organization.
3. Build a bot that enables permitted users to send targeted or organization-wide messages.
4. Develop an interactive bot for end-users to provision their own devices using a MAC address.
5. Integrate API calls, handle pagination, and manage bot interactions for real-world scenarios.

Exercise 1: Create a feedback bot for your organization

Develop an assistant bot that allows only permitted users to collect feedback from all users in the organization.

Requirements:

1. Only allow permitted users to access the bot's feedback functionality.
2. Provide access through the default card to send a feedback request to all users in the organization.
3. Present an Adaptive Card with a text input box for the user to enter the feedback request message.
4. Collect responses from recipients and send a confirmation or error notification to the requesting user after feedback is submitted.



▲ **Warning:** In this exercise, you will need to add the **spark-admin:people_read** scope to your Service App. Please notify us to have your Service App rights approved.

Pagination

When retrieving a large amount of information, you need to handle pagination. With pagination, the API returns a limited set of results per request. If more data is available, the response includes a link labeled `rel="next"`, which provides the URL to fetch the next page of results. This process continues until all results have been retrieved.

1. Navigate to `06-usecases/01_pagination.py` and review the code.

In this example, a pagination limit of 2 users is used.

2. Execute the code with the following command and let it run:

- `python 01_pagination.py`

3. You will now see all the users listed in the console, displayed in pages of size 2:

```
DEBUG: Fetching page 1 of people.
DEBUG: Making API request to URL: https://webexapis.com/v1/people?max=2
DEBUG: API request successful. Status Code: 200
Printing people from page 1:
Name: Anita Perez, Email: ['aperez@cb136.dc-03.com']
Name: Charles Holland, Email: ['cholland@cb136.dc-03.com']
DEBUG: Found 'next' pagination link. Next URL: https://webexapis.com/v1/people?max=2&cursor=bGltaXQ9MiZzdGFydEluZGV4PTM

DEBUG: Fetching page 2 of people.
DEBUG: Making API request to URL: https://webexapis.com/v1/people?max=2&cursor=bGltaXQ9MiZzdGFydEluZGV4PTM
DEBUG: API request successful. Status Code: 200
Printing people from page 2:
Name: Eric Steele, Email: ['esteele@cb136.dc-03.com']
Name: Kellie Melby, Email: ['kmelby@cb136.dc-03.com']
DEBUG: Found 'next' pagination link. Next URL: https://webexapis.com/v1/people?max=2&cursor=bGltaXQ9MiZzdGFydEluZGV4PTU

DEBUG: Fetching page 3 of people.
DEBUG: Making API request to URL: https://webexapis.com/v1/people?max=2&cursor=bGltaXQ9MiZzdGFydEluZGV4PTU
DEBUG: API request successful. Status Code: 200
Printing people from page 3:
Name: Rebekah Barretta, Email: ['rbarretta@cb136.dc-03.com']
Name: Ricardo Filice, Email: ['rfilice@cb136.dc-03.com']
DEBUG: Found 'next' pagination link. Next URL: https://webexapis.com/v1/people?max=2&cursor=bGltaXQ9MiZzdGFydEluZGV4PTc

DEBUG: Fetching page 4 of people.
DEBUG: Making API request to URL: https://webexapis.com/v1/people?max=2&cursor=bGltaXQ9MiZzdGFydEluZGV4PTc
DEBUG: API request successful. Status Code: 200
Printing people from page 4:
Name: Stefan Mauk, Email: ['smauk@cb136.dc-03.com']
Name: Taylor Bard, Email: ['tbard@cb136.dc-03.com']
DEBUG: No more 'next' pagination links found. All people listed.
DEBUG: No 'next' pagination link found. All people have been listed.
```

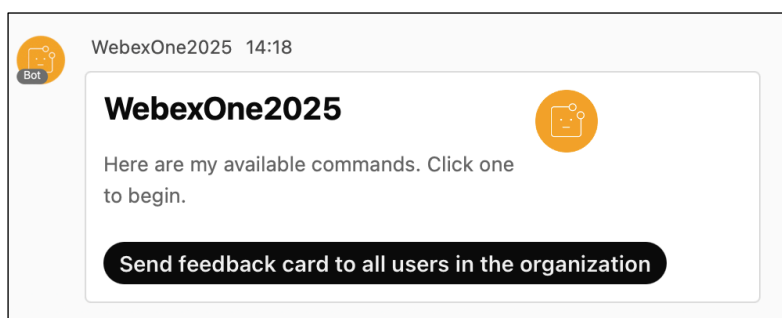
Solution

1. Navigate to 06-usecases/01_feedback.py and review the code.

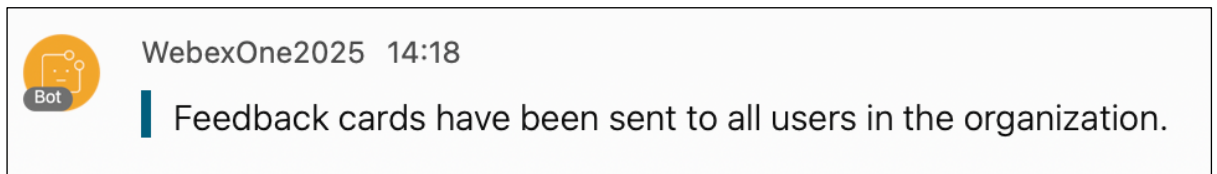
These are the key points to understand how each requirement has been met:

1. **Permitted User Access Only**
The `is_allowed_sender` function verifies the user's email against a predefined constant before allowing command execution.
 2. **Org-wide Feedback Card Distribution**
The `/feedback` command sends an Adaptive Card containing a feedback form to every user in the Webex organization.
 3. **Adaptive Card Feedback Input**
The Adaptive Card created by the bot includes a multi-line `Input.Text` element specifically for feedback entry.
 4. **Collect Responses, Notify Sender**
A chained command extracts the feedback from the submitted card, forwards it to the designated email, and sends a confirmation or error message back to the submitting user.
2. Change your directory in the terminal:
 - `cd ../06-usecases`
 3. Execute the code with the following command and let it run:
 - `python 01_feedback.py`

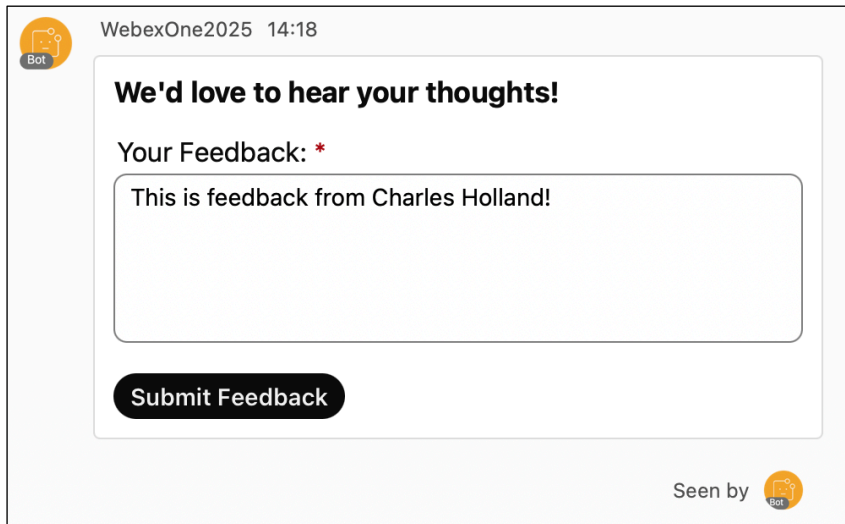
▲ **Warning:** Wait until you see the message “WebSocket Opened.” appear in the console.
 4. Send any message to your bot to get the following card:



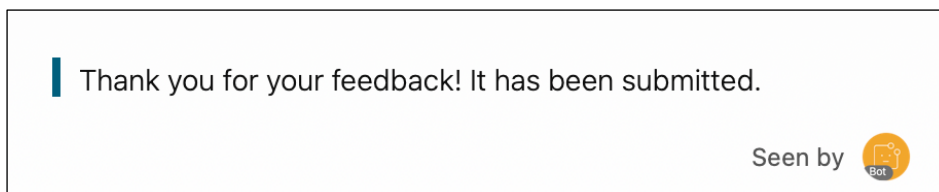
5. Once you click **Send feedback card to all users in the organization**, the cards will begin sending. You will receive the following notification message once the process is completed:



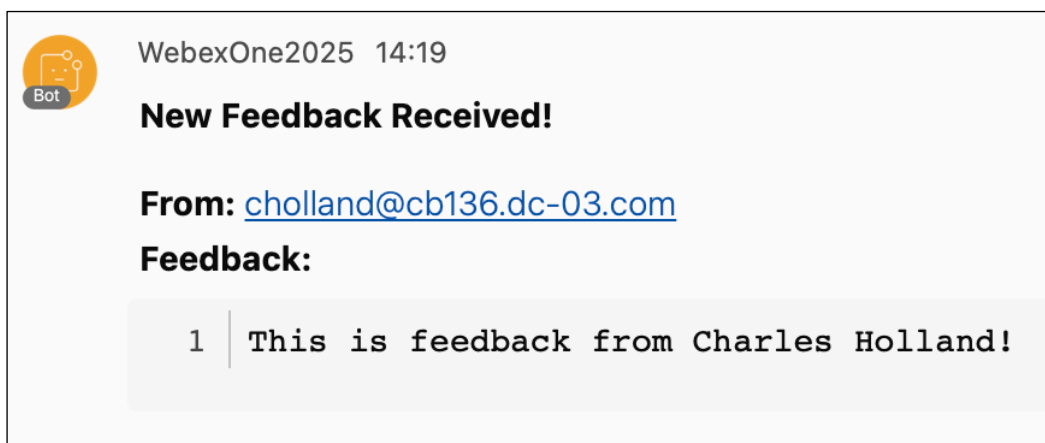
6. At this point, all users will receive the following feedback card:



7. Once users submit their feedback, they will receive the following notification message:



8. At the same time, you, as an admin, will be notified with the user's feedback:

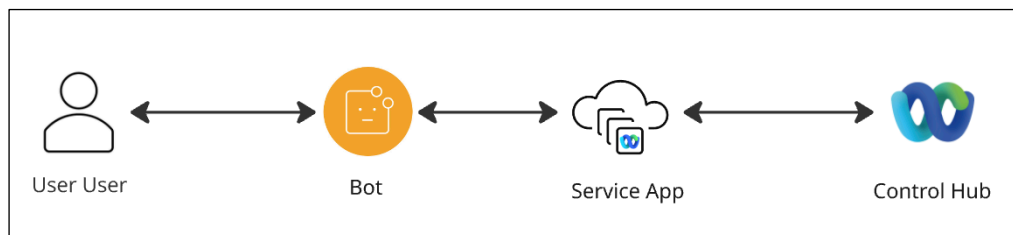


Exercise 2: End-User Phone Provisioning Assistant

Create an interactive Webex bot that enables end-users to auto-provision their new phones by submitting the device's MAC address. The bot will guide the user through the process using an Adaptive Card and interact with backend services to complete the provisioning.

Requirements:

1. Validate that the MAC address provided by the user follows the correct format (e.g., AA:BB:CC:DD:EE:FF or AABBCCDDEEFF).
2. Present an Adaptive Card to the user with:
 1. An input field for the MAC address.
 2. A choice set (dropdown) for selecting the phone model, with the following pre-defined options:
 - DMS Cisco 8851
 - DMS Cisco 8861
 - DMS Cisco 8865
3. Use the submitted data (MAC address and selected phone model) to initiate device provisioning through the backend or relevant API.
4. Notify the user in Webex of the provisioning result (success or error).



▲ **Warning:** In this exercise, you will need to add the **spark-admin:devices_write** scope to your Service App. Please notify us to have your Service App rights approved.

Solution

1. Navigate to 06-usecases/02_device.py and review the code.

These are the key points to understand how each requirement has been met:

- 1. MAC address validation**

The `is_valid_mac_address` function uses regex to check the MAC format before API submission.

- 2. Adaptive Card presentation**

The `AutoProvisioning` command displays an Adaptive Card with MAC input and a phone model dropdown.

- 3. Device provisioning**

The `ProvisionCallback` extracts card data and POSTs it to the `/v1/devices` API for provisioning.

- 4. Provisioning result notification**

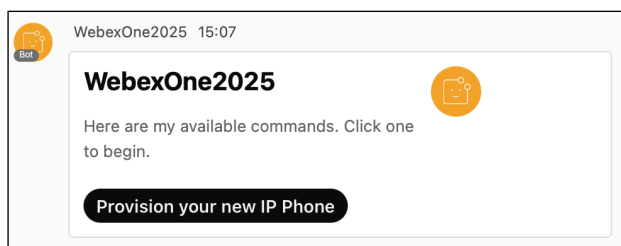
`ProvisionCallback` returns `quote_info` messages (success, duplicate, or error) based on the API's response status.

2. Execute the code with the following command and let it run:

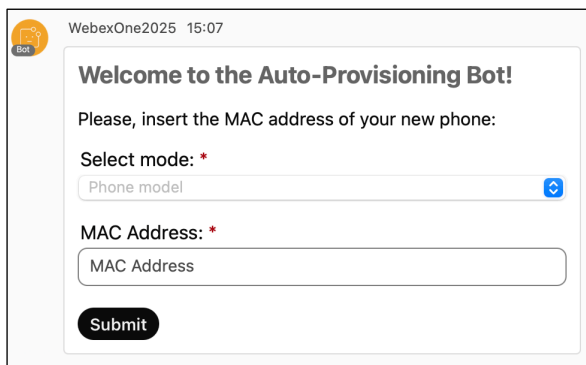
- `python 01_feedback.py`

⚠ Warning: Wait until you see the message “WebSocket Opened.” appear in the console.

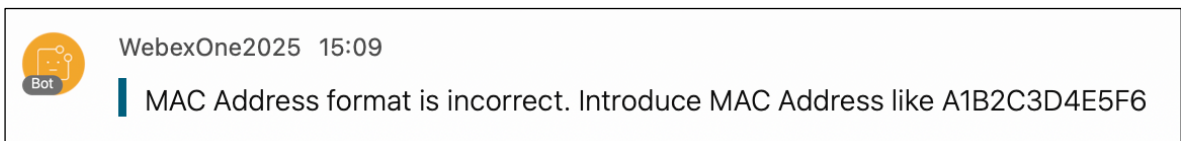
3. Send any message to your bot to get the following card:



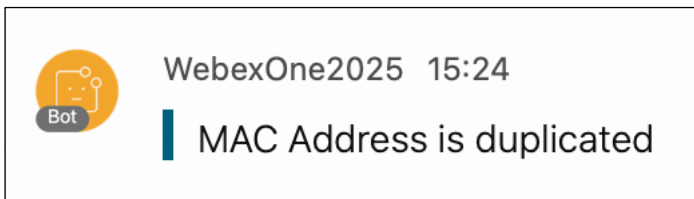
4. Click **Provision your new IP Phone** to get the following provisioning card:

A screenshot of a Webex chat window. At the top, it says 'WebexOne2025 15:07'. Below that is a message from the bot 'WebexOne2025' with a small orange bot icon. The message text is 'Welcome to the Auto-Provisioning Bot!' followed by 'Please, insert the MAC address of your new phone:'. There is a form with two fields: 'Select mode: *' with a dropdown menu showing 'Phone model' and a blue arrow icon, and 'MAC Address: *' with a text input field. At the bottom of the form is a black button with white text that says 'Submit'.

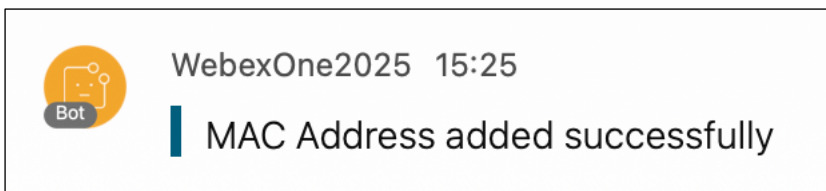
5. If you submit a MAC address in the wrong format, you will receive a notification about the error:



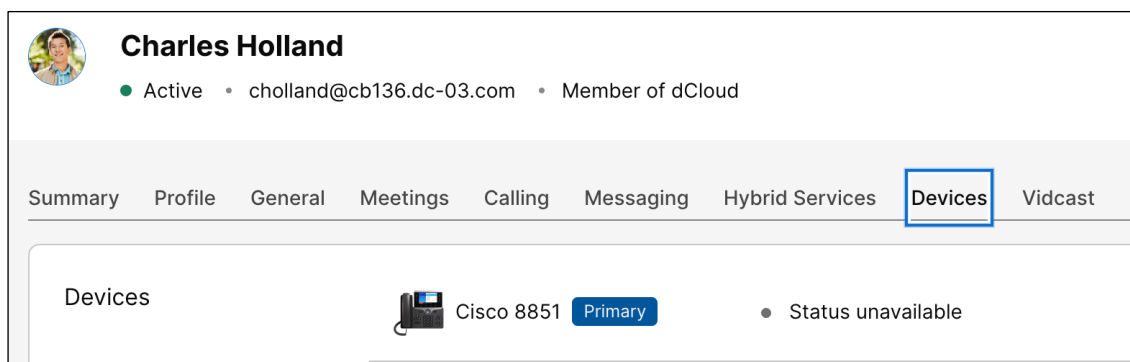
6. If you submit a duplicate MAC address, such as "54A3152300C8", you will receive a notification about the duplication:



7. If you submit a valid MAC address, you will receive a message like the following:



8. You can verify the device registration in Control Hub:



6 – Troubleshooting

During this section, you will be presented with different exercises to apply the concepts you learn and use different Webex APIs to get relevant information for troubleshooting.

You can use any of the tools—either Postman, Webex for Developers, or Python scripts—to resolve them.

You will also be presented with the steps for resolution.

Upon completion of this section, you will be able to:

1. Check general Webex Status
2. Check different Audit events
3. Create reports and download them
4. Check Events as a Compliance Officer
5. Get Call History and statistics
6. Check Meeting statistics

Reference:

<https://developer.webex.com/>

Exercise 1: Webex Status API

In this exercise, you will use the Webex Status API to retrieve and interpret key information about the current operational status of Webex services.

This API can be useful for quickly identifying service outages or degradations, troubleshooting reported issues, informing users about ongoing incidents, and deciding when to escalate or communicate about disruptions.

1. Get the general Webex status indicator.
2. Check if there are any current unresolved incidents.
3. Check if there is any ongoing maintenance.

References:

<https://developer.webex.com/calling/docs/webex-status-api>

<https://status.webex.com/api/>

Solution

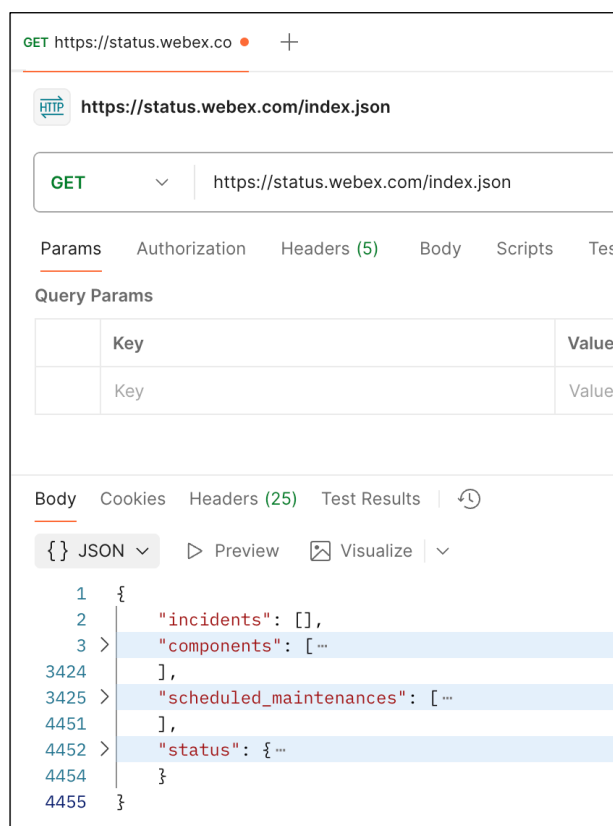
In this section, you do not need an access token to access the Webex Status API.

Similarly, you cannot access this information through the Webex Developer Portal, so you will need to create a query using either Postman or Python.

To get a full overview of available status endpoints and summary information, perform a GET request to:

<https://status.webex.com/index.json>

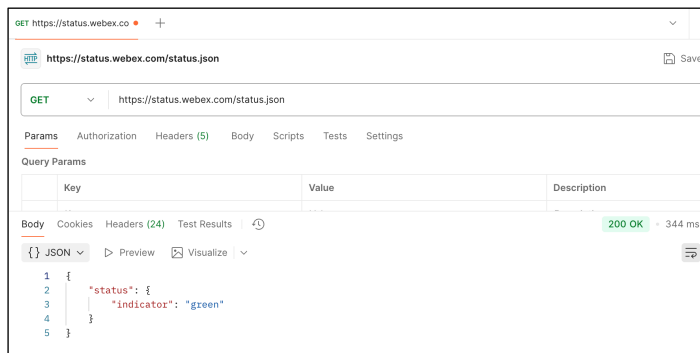
This endpoint provides a summary of the overall status, unresolved incidents, and both active and upcoming scheduled maintenances. It serves as a central resource for quickly accessing all key status information in one response.



1. Indicator

To get the status indicator, perform a GET request to:

<https://status.webex.com/status.json>



The possible values are green, yellow, and red, which indicate the overall status of Webex services:

- Green: All systems operational
- Yellow: Degraded performance or partial outage
- Red: Major outage

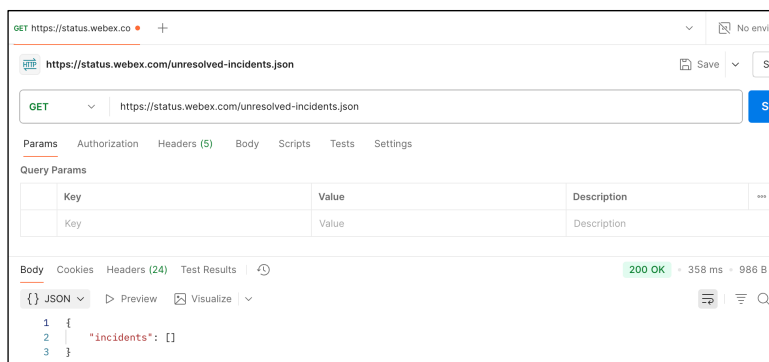
In this case, there are no issues reported.

2. Incidents

To check for any current unresolved incidents, perform a GET request to:

<https://status.webex.com/unresolved-incidents.json>

This endpoint returns a list of incidents that are not yet resolved.



In this case, there are no incidents reported at this time.

3. Active Maintenance:

To check for any ongoing maintenance activities, perform a GET request to:
<https://status.webex.com/active-scheduled-maintenances.json>

This endpoint returns a list of all currently active scheduled maintenances

The screenshot shows a REST client interface with the following details:

- Request:** GET `https://status.webex.com/active-scheduled-maintenances.json`
- Response:** 200 OK, 393 ms
- Body:** JSON format showing an empty array:

```
1 {  
2   "scheduled_maintenances": []  
3 }
```

In this case, there are no active maintenance activities at this time.

Exercise 2: Admin Audit Events

In this exercise, you need to access the Admin Audit Events to identify different types of events. If you run this with your current user, you will receive a "Forbidden" error.

You can use the following credentials for this activity:

User	Username
esteele@cb127.dc-02.com	dCloud2856!

During this exercise, you need to:

1. List all admin audit event categories available
2. Get all the logins from the last 5 days
3. Get all the organization configuration changes for the last 5 days

Reference:

<https://developer.webex.com/admin/docs/api/v1/admin-audit-events>

Solution

1. List admin audit categories:

To list all available admin audit event categories, send a GET request to:

<https://webexapis.com/v1/adminAuditEventCategories>

The screenshot displays a REST client interface with two main panels. The left panel, titled 'Admin Audit Events', shows the endpoint 'List Admin Audit Event Categories' with the operation ID 'List_Admin_Audit_Event_Categories' and description 'Get the list of all admin event categories.' The request is a GET to '/adminAudit/eventCategories'. It shows a successful response with status 200 and an 'OK' message. Below this, it shows a failed response with status 400 and a message: 'Bad Request: The request was invalid or cannot be otherwise served. An accompanying error message will explain further.' The right panel, titled 'Configuration', shows the same endpoint and request method. It includes sections for 'Query Params', 'Headers', and 'Authorization'. The 'Authorization' section is set to 'Bearer' with a token field. The 'Accept' header is set to 'application/json'. A 'Run' button is visible at the bottom of the configuration panel. The response section shows 'Response: 200' with tabs for 'Data' and 'Info'.

This will give you an overview of all the types of admin activities you can monitor and query, helping you understand the full range of auditing possibilities available.

2. Get all the logins:

To retrieve all login events from the last 5 days, send a GET request to the admin audit events endpoint with the following parameters:

- from: 2025-09-24T00:00:00.000Z
- to: 2025-09-29T00:00:00.000Z
- orgId: cf0be353-2dd4-463f-9843-4a3c27a9a9d3
- eventCategories: LOGINS

```

{
  "items": [
    {
      "data": {
        "actorOrgName": "Atlas_Test_cb127.dc-02.",
        "targetName": "Y2YwYmUzNTMtMmRkNC00NjNmL",
        "eventDescription": "An Admin logged in",
        "actorName": "Eric Steele",
        "actorEmail": "esteele@cb127.dc-02.com",
        "adminRoles": [
          "User",
          "Full_Admin",
          "atlas-portal.troubleshooting.advanc",
          "cdr-rest-svr.read_org_cdrs",
          "id_full_admin"
        ],
        "trackingId": "ATLAS_f318aeda-7c68-45b9-1",
        "targetType": "ORG",
        "targetId": "Y2YwYmUzNTMtMmRkNC00NjNmLTk",
        "actorUserAgent": "Mozilla/5.0 (Macintosh",
        "eventCategory": "LOGINS",
        "actorIp": "12.157.146.250",
        "targetOrgId": "Y2YwYmUzNTMtMmRkNC00NjNmL",
        "actionText": "Eric Steele logged into o",
        "targetOrgName": "Atlas_Test_cb127.dc-02",
      },
      "created": "2025-09-28T16:40:48.089Z",
      "actorOrgId": "Y2YwYmUzNTMtMmRkNC00NjNmLTk4NI",
      "id": "M2E1Y2Q4YTYtYTBjOC00NDdhLWE5MDEt0GE4MI",
      "actorId": "MjJlNjhjMzAtY2YzMzNC00YmM2LWE5ZjQ0"
    }
  ]
}

```

3. Organization changes

To retrieve all organization configuration change events from the last 5 days, send a GET request to the admin audit events endpoint with the following parameters:

- from: 2025-09-24T00:00:00.000Z
- to: 2025-09-29T00:00:00.000Z
- orgId: cf0be353-2dd4-463f-9843-4a3c27a9a9d3
- eventCategories: ORG_SETTINGS

```

{
  "items": [
    {
      "data": {
        "actorOrgName": "Atlas_Test_cb127.dc-02.com",
        "targetName": "Eric Steele",
        "operationType": "CREATE",
        "eventDescription": "An org setting was created",
        "actorName": "Eric Steele",
        "actorEmail": "esteele@cb127.dc-02.com",
        "settingKey": "terms-of-service-user-info",
        "settingName": "terms-of-service-user-info",
        "settingValue": "{\"acceptedDateTimeStamp\": \"2025-09-28T16:40:52.489Z\", \"trackingId\": \"ATLAS_f54887c3-1cbb-42ca-8100-000000000000\", \"previousValue\": \"Null\", \"targetType\": \"PERSON\", \"targetId\": \"MjJlNjhjMzAtY2MzNC00YmM2LWE5ZjQ0OTk4NDk1\", \"actorUserAgent\": \"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7; rv:109.0) Gecko/20100101 Firefox/115.0\", \"eventCategory\": \"ORG_SETTINGS\", \"actorIp\": \"12.157.146.250\", \"targetOrgId\": \"Y2YwYmUzNTMtMmRkNC00NjNmLTk4NDk1\", \"actionText\": \"Eric Steele has modified the terms of service user info\", \"entityType\": \"USER\", \"targetOrgName\": \"Atlas_Test_cb127.dc-02.com\"}",
        "created": "2025-09-28T16:40:52.489Z",
        "actorOrgId": "Y2YwYmUzNTMtMmRkNC00NjNmLTk4NDk1",
        "id": "ZjNhZDY0NzU0MGY2ZS00ZTBiLTk3ZDI0MGZlNjQ0OTk4NDk1",
        "actorId": "MjJlNjhjMzAtY2MzNC00YmM2LWE5ZjQ0OTk4NDk1"
      }
    },
    {
      "data": {
        "actorOrgName": "Atlas_Test_cb127.dc-02.com",
        "targetName": "Eric Steele",
        "operationType": "UPDATE",
        "eventDescription": "An org setting was updated",
        "actorName": "Eric Steele",
        "actorEmail": "esteele@cb127.dc-02.com",
        "settingKey": "terms-of-service-user-info",
        "settingName": "terms-of-service-user-info",
        "settingValue": "{\"acceptedDateTimeStamp\": \"2025-09-28T16:40:52.489Z\", \"trackingId\": \"ATLAS_f54887c3-1cbb-42ca-8100-000000000000\", \"previousValue\": \"Null\", \"targetType\": \"PERSON\", \"targetId\": \"MjJlNjhjMzAtY2MzNC00YmM2LWE5ZjQ0OTk4NDk1\", \"actorUserAgent\": \"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7; rv:109.0) Gecko/20100101 Firefox/115.0\", \"eventCategory\": \"ORG_SETTINGS\", \"actorIp\": \"12.157.146.250\", \"targetOrgId\": \"Y2YwYmUzNTMtMmRkNC00NjNmLTk4NDk1\", \"actionText\": \"Eric Steele has modified the terms of service user info\", \"entityType\": \"USER\", \"targetOrgName\": \"Atlas_Test_cb127.dc-02.com\"}",
        "created": "2025-09-28T16:40:52.489Z",
        "actorOrgId": "Y2YwYmUzNTMtMmRkNC00NjNmLTk4NDk1",
        "id": "ZjNhZDY0NzU0MGY2ZS00ZTBiLTk3ZDI0MGZlNjQ0OTk4NDk1",
        "actorId": "MjJlNjhjMzAtY2MzNC00YmM2LWE5ZjQ0OTk4NDk1"
      }
    }
  ]
}

```

Exercise 3: Reports

Create and check a report for **User Activity Summary**.

1. Use the Reports API to create a report with the type `user_activity_summary`.
2. Monitor the status of the report until it is complete.
3. Download and review the report once it's ready.

Reference:

<https://developer.webex.com/docs/api/v1/reports>

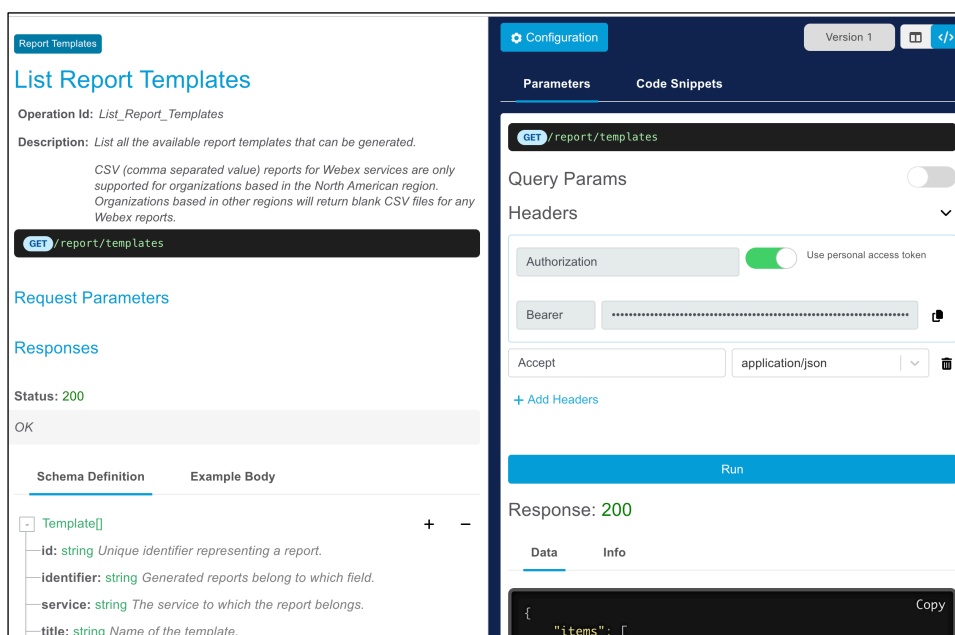
Solution

To complete this activity, follow these four steps:

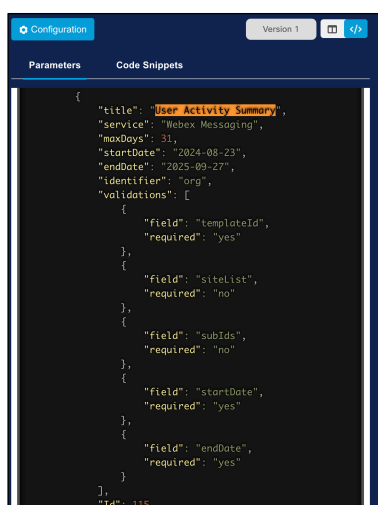
1. Get the report templates to identify the desired report.
2. Create the report.
3. Retrieve the download link for the report.
4. Download the report.

Let's walk through each step:

1. First of all, you need to get the report templates to find the report and its Id:
<https://developer.webex.com/admin/docs/api/v1/report-templates/list-report-templates>



Look for the report “User Activity Summary”:



It has the Id: 115

2. Now that you have the report ID, you can generate the report.

The `startDate` and `endDate` fields are mandatory. For example, to generate the report for the whole month of September, use:

- `startDate: 2025-09-01`
- `endDate: 2025-09-28`

Reports

Create a Report

Operation Id: `Create_a_Report`

Description: Create a new report. For each `templateId`, there are a set of validation rules that need to be followed. For example, for templates belonging to Webex, the user needs to provide `siteUrl`. These validation rules can be retrieved via the [Report Templates API](#).

The `'templateId'` parameter is a number. However, it is a limitation of `developer.webex.com` platform that it is passed as a string when you try to test the API from here.

CSV reports for Teams services are only supported for organizations based in the North American region. Organizations based in a different region will return blank CSV files for any Teams reports.

POST /reports

Request Parameters

Request body

Schema Definition **Example Body**

`object`

Configuration Version 1

Parameters **Code Snippets**

POST /reports

Request Body (Form) **Request Body (JSON)**

templateId*
Unique ID representing valid report templates.

startDate
Data in the report will be from this date onwards.

endDate
Data in the report will be until this date.

siteList
Sites belonging to user's organization. This attribute is needed for site-based

After submitting your request, you will receive the report ID in the response.

Response: 200

Data **Info**

```
{
  "items": {
    "Id": "Y2lZy29zcGFyazovL3VzL1JFUE9SVC9jZmE3NWMxN"
  }
}
```

Copy

3. Using the previously created report ID, get the report details:
<https://developer.webex.com/admin/docs/api/v1/reports/get-report-details>

Get Report Details

Operation Id: `Get_Report_Details`

Description: Shows details for a report, by report ID.

Specify the report ID in the `reportId` parameter in the URL.

Notes:
CSV reports for Teams services are only supported for organizations based in the North American region. Organizations based in a different region will return blank CSV files for any Teams reports.

Reports are usually provided in zip format. A Content-header `application/zip` or `application/octet-stream` does indicate the zip format. There is usually no .zip file extension.

```
GET /reports/{reportId}
```

Request Parameters

Path

reportId required | `string`

The unique identifier for the report.

Configuration

Version 1

Parameters

```
GET /reports/Y21zY29zcGFyazovL3VzL1JFUE9SVC9jZmE3NmMxNjgxMzY0NGVlODQwNmM1MjJjYWI3ZTljOQ
```

reportId*
The unique identifier for the report.

Y21zY29zcGFyazovL3VzL1JFUE9SVC9jZmE3NmMxNjgxMzY0NGVlODQwNmM1

Query Params

☐

Headers

☐ Use personal access token

Authorization

Bearer

Accept application/json

[+ Add Headers](#)

In the response, you will find the downloadURL for your report:

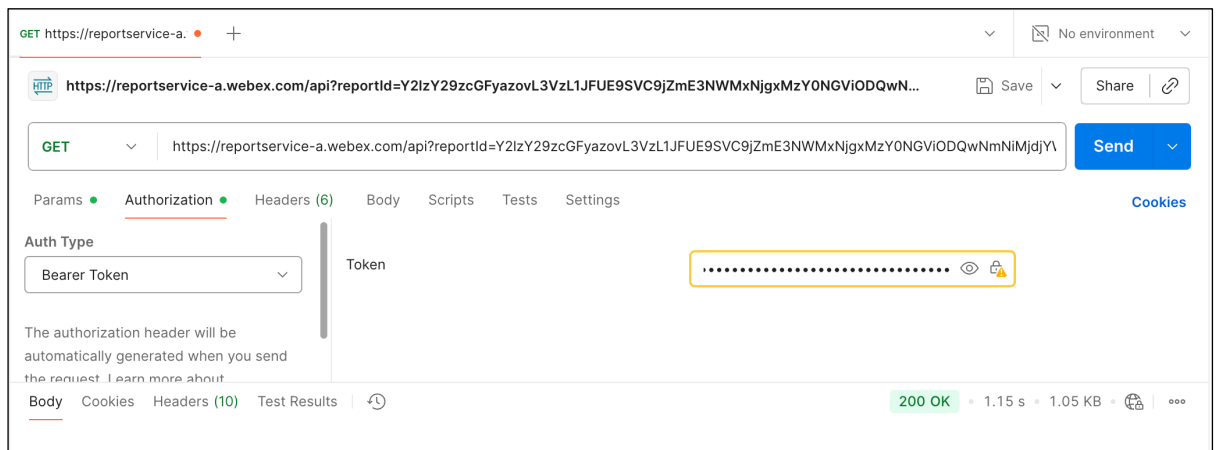
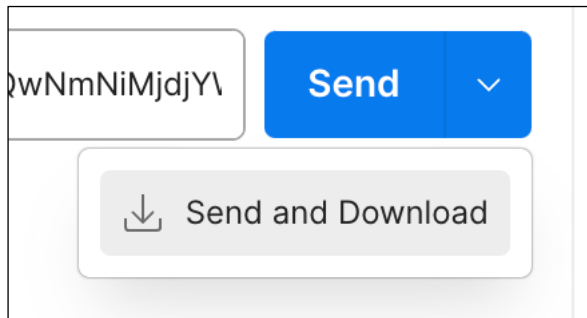
Response: 200

Data **Info**

```
{
  "items": [
    {
      "title": "User Activity Summary",
      "service": "Webex Messaging",
      "startDate": "2025-09-01",
      "endDate": "2025-09-27",
      "siteList": "null",
      "created": "2025-09-28 16:32:56",
      "createdBy": "Y21zY29zcGFyazovL3VzL1BFT1BMRS",
      "scheduleFrom": "api",
      "status": "done",
      "downloadDomain": "https://reportservice-a.w",
      "downloadURL": "https://reportservice-a.webe",
      "Id": "Y21zY29zcGFyazovL3VzL1JFUE9SVC9jZmE3N"
    }
  ],
  "numberOfReports": 1
}
```

Copy

4. To download the report, you can use the "Send and Download" option in Postman with the downloadURL you received:



Exercise 4: Events

To access the Events API, you need to use a user account with the "Compliance Officer" role. If you do not have this role, you will receive the following error when trying to use the API:

- "Compliance role required to get events."

Functional	☐ Support admin ⓘ
	☐ User and device admin ⓘ
	☐ Device admin ⓘ
	☒ Compliance officer ⓘ
	☐ Advanced troubleshooting access ⓘ
	☐ Webex Calling Detailed Call History API access ⓘ

You can use the following credentials for this activity:

User	Username
aperez@cb127.dc-02.com	dCloud2856!

You have to:

1. Use the Events API to retrieve events filtered by the resource memberships.

Reference:

<https://developer.webex.com/admin/docs/api/v1/events>

Solution

You can list all the events from this API

<https://developer.webex.com/admin/docs/api/v1/events/list-events>

The screenshot shows the Webex API documentation for the `GET /events` endpoint. On the left, the 'List Events' section includes the operation ID `List_Events`, a description of listing events with filterable query parameters, and a note that long result sets are split into pages. Below this, the 'Request Parameters' section shows the query parameter `resource` with a value of `string` and an example of filtering by resource type like 'messages'. On the right, the 'Configuration' panel shows the 'Parameters' tab for `GET /events`. It lists query parameters: `resource` (set to 'memberships'), `type` (set to 'created'), `actorId` (set to a UUID), `from` (set to a timestamp), `to` (set to a timestamp), and `max` (set to 100). A '+ Add Query Parameters' link is at the bottom.

To track activity related to room memberships—such as when users join or leave rooms—filter by the resource memberships.

This will show all events involving changes to room participation.

The screenshot shows the API response for the `GET /events` endpoint. The response status is `200`. The 'Data' tab is selected, showing a JSON array of event items. The first item is a 'created' event for the 'memberships' resource, triggered by a user named Charles Holland. The event data includes the room ID, room type (group), person ID, person email, person display name, person org ID, and whether the person is a moderator, monitor, or room hidden.

```
{
  "items": [
    {
      "id": "Y2lzY29zcGFyazovL3VybjpURUFN0nVzLXd1c:",
      "resource": "memberships",
      "type": "created",
      "actorId": "Y2lzY29zcGFyazovL3VzL1BFT1BMRS8w",
      "created": "2025-09-25T09:35:52.025Z",
      "data": {
        "id": "Y2lzY29zcGFyazovL3VybjpURUFN0nVzL",
        "roomId": "Y2lzY29zcGFyazovL3VybjpURUFN0",
        "roomType": "group",
        "personId": "Y2lzY29zcGFyazovL3VzL1BFT1B",
        "personEmail": "cholland@cb127.dc-02.com",
        "personDisplayName": "Charles Holland",
        "personOrgId": "Y2lzY29zcGFyazovL3VzL09SI",
        "isModerator": false,
        "isMonitor": false,
        "isRoomHidden": false,
        "created": "2025-09-25T09:35:52.025Z"
      }
    },
    {
      "id": "Y2lzY29zcGFyazovL3VybjpURUFN0nVzLXd1c:",
      "resource": "memberships",

```

Exercise 5: Calling Troubleshooting

In this exercise, you will work with the Webex Calling APIs to troubleshoot and review call activity.

You will need to use a user account with the “Webex Calling Detailed Call History API access” role:

Administrator roles	Organizational	☒ Organization admin
		☐ Full admin ⓘ ☐ Read-only admin ⓘ
	Functional	☒ Support admin ⓘ ☒ User and device admin ⓘ ☒ Device admin ⓘ ☐ Compliance officer ⓘ ☒ Advanced troubleshooting access ⓘ ☒ Webex Calling Detailed Call History API access ⓘ

You can use the following credentials for this activity:

User	Username
esteele@cb127.dc-02.com	dCloud2856!

References:

<https://developer.webex.com/blog/webex-detailed-call-history-api>

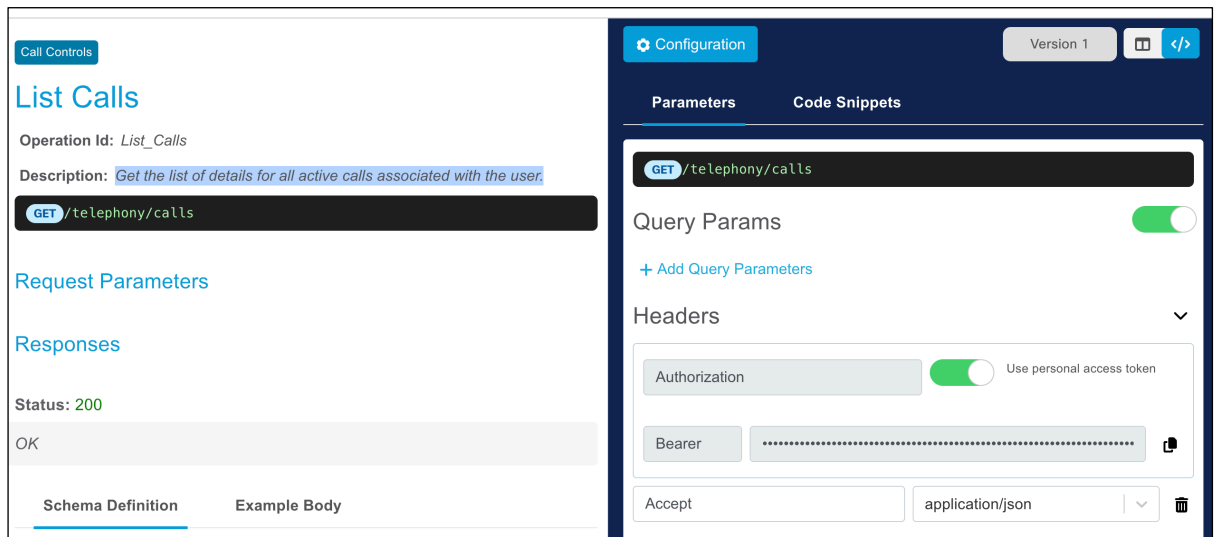
<https://developer.webex.com/blog/exploring-the-webex-calling-reports-and-analytics-apis>

<https://developer.webex.com/calling/docs/api/v1/reports-detailed-call-history/get-detailed-call-history>

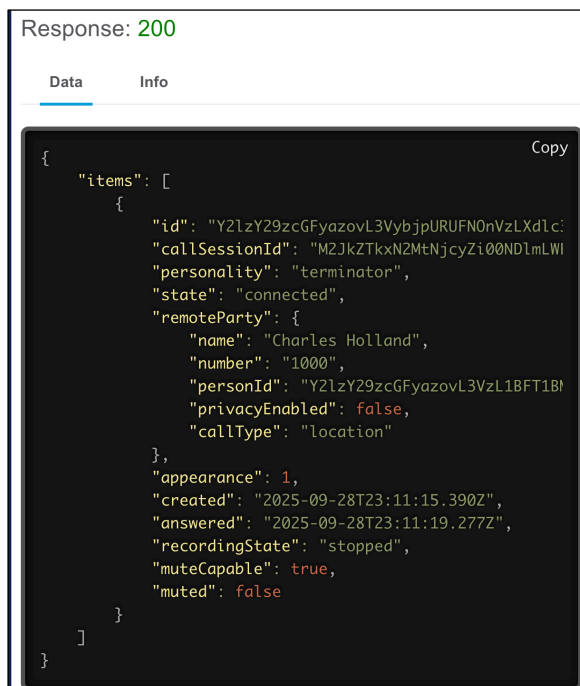
Solution

Before checking the detailed call history, you will explore the following Calling APIs related to calls:

- List Calls: Retrieves details for all active calls associated with the user
<https://developer.webex.com/calling/docs/api/v1/call-controls/list-calls>



You may not see any calls at the moment, but this is how the response will look:



- List Call History: Retrieves up to 20 call history records for the user.
<https://developer.webex.com/calling/docs/api/v1/call-controls/list-call-history>

Call Controls

List Call History

Operation Id: `List_Call_History`

Description: Get the list of call history records for the user. A maximum of 20 call history records per type (`placed` , `missed` , `received`) are returned.

GET `/telephony/calls/history`

Request Parameters

Query

type | `string`

The type of call history records to retrieve. If not specified, then all call history records are retrieved.

enum = ["placed", "missed", "received"]

Responses

Configuration Version 1

Parameters **Code Snippets**

GET `/telephony/calls/history`

Query Params

☐ `type` `placed` ☐

[+ Add Query Parameters](#)

Headers

Authorization ☒ Use personal access token

Bearer `.....` ☐

Accept `application/json` ☐

This response won't contain much detailed information about the calls.

Response: 200

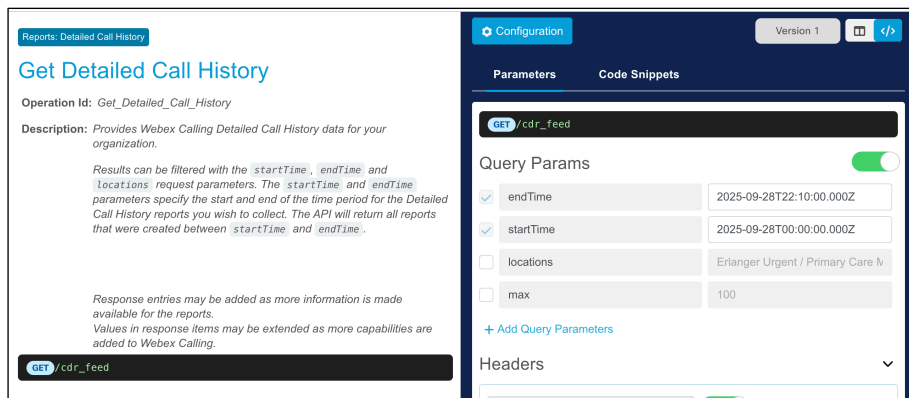
Data **Info**

```
{
  "items": [
    {
      "type": "received",
      "name": "Charles Holland",
      "number": "1000",
      "privacyEnabled": false,
      "time": "2025-09-28T16:11:19.278-07:00"
    },
    {
      "type": "received",
      "name": "Charles Holland",
      "number": "1000",
      "privacyEnabled": false,
      "time": "2025-09-28T16:09:05.703-07:00"
    },
    {
      "type": "placed",
      "name": "Charles Holland",
      "number": "+16692810845",
      "privacyEnabled": false,
      "time": "2025-09-28T16:05:04.829-07:00"
    }
  ]
}
```

Copy

These APIs provide only basic information and are limited to the user making the API call.

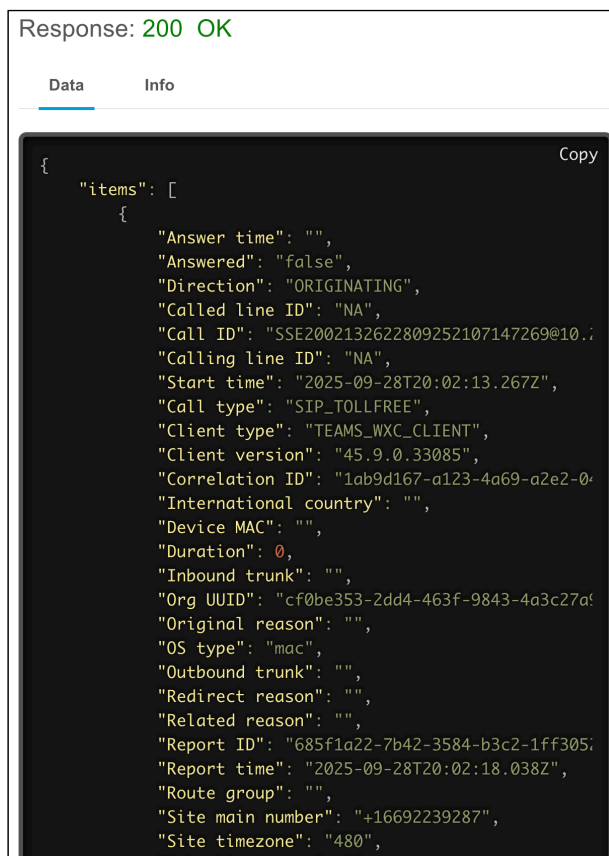
1. Detail Call history - Provides Webex Calling Detailed Call History data for your organization.
<https://developer.webex.com/calling/docs/api/v1/reports-detailed-call-history/get-detailed-call-history>



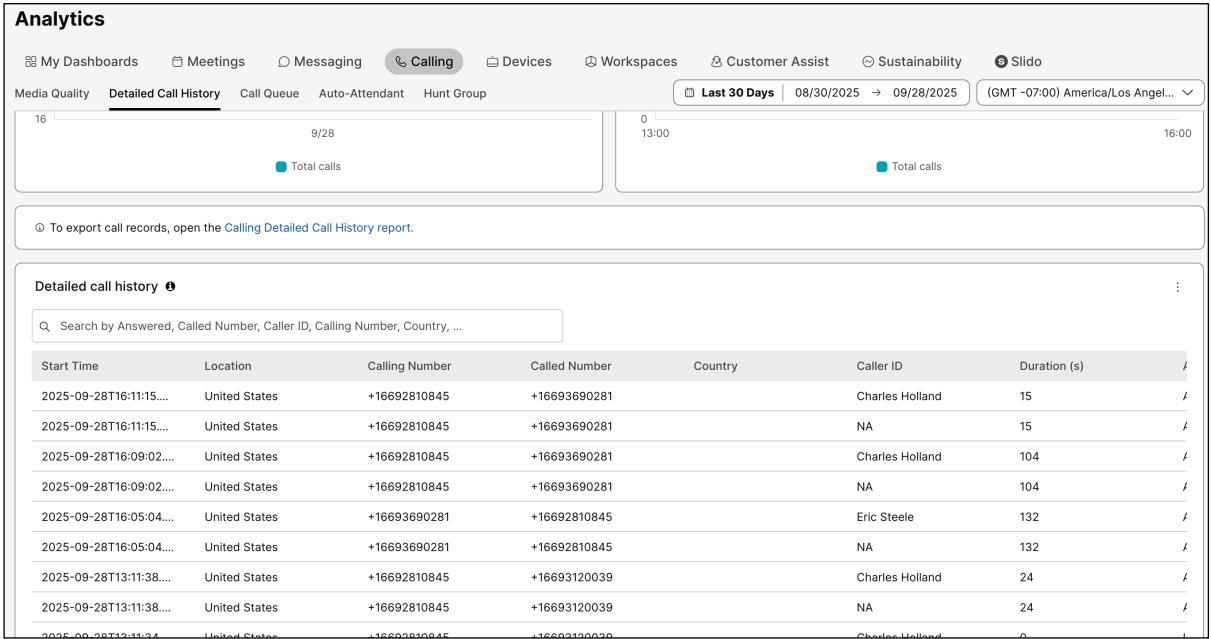
Use the following parameters for your request:

- endTime: 2025-09-28T22:10:00.000Z
- startTime: 2025-09-28T00:00:00.000Z

The result contains very detailed information about each call.



The information provided by this call is equivalent to the information provided in CH:



The response from the Webex Calling Detailed Call History API includes:

- **Call direction:** Originating or terminating
- **Call type:** For example, SIP_ENTERPRISE or SIP_NATIONAL
- **Timestamps:** Start time, release time, and answer time
- **User information:** Name, UUID, and user type
- **Calling and called numbers:** Both internal and external numbers
- **Device and client details:** Device MAC, OS type, client type, and version
- **Call metrics:** Ring duration, call duration, and answer indicator
- **Call outcome:** Success, refusal, or temporarily unavailable, with outcome reasons provided
- **Session and network data:** Session IDs, correlation IDs, and location information

This detailed information helps you analyze user activity, troubleshoot call issues, trace call flows, and identify patterns or recurring problems within your organization.

Exercise 6: Meetings Troubleshooting

In this exercise, you will use the Meetings APIs to retrieve meeting details and analyze meeting quality metrics.

You have to:

1. Get Meeting IDs for your organization's meetings.
2. Retrieve meeting quality data for a specific meeting.

You can use the following credentials for this activity:

User	Username
esteele@cb127.dc-02.com	dCloud2856!

References:

<https://developer.webex.com/meeting/docs/api/v1/meetings/list-meetings>

<https://analytics.webexapis.com/v1/meeting/qualities?meetingId=<meetingId>>

▲ **Warning:** You need to use Postman to access Meeting Quality data.

Solution

1. Get MeetingId:

Use the following endpoint to list meetings and obtain the Meeting ID:

<https://developer.webex.com/meeting/docs/api/v1/meetings/list-meetings>

Make sure to filter by meetingType set to "meeting".

The screenshot shows the Webex API documentation for the 'List Meetings' endpoint. On the left, there's a description of the endpoint, noting that it retrieves details for meetings with a specified meeting number, web link, meeting type, etc. It also mentions that the previous 'List Meetings of a Meeting Series' API is merged into this one. On the right, there's a 'Configuration' panel with 'Parameters' and 'Code Snippets' tabs. The 'Parameters' tab is active, showing a table of query parameters for the 'GET /meetings' endpoint. The parameters include meetingNumber, webLink, roomId, meetingSeriesId, max, from, to, meetingType (checked), state, scheduledType, and isModified. The meetingType is set to 'meeting'.

Parameter	Value
meetingNumber	123456789
webLink	https%3A%2F%2Fgo.webex.com
roomId	Y2lzY29zcGFyZ3VzL1JPT0I
meetingSeriesId	25bbf831-5be9-4c25-b4b0-9b59
max	100
from	2019-03-18T09:30:00+08:00
to	2019-03-25T09:30:00+08:00
meetingType	meeting
state	InProgress
scheduledType	personalRoomMeeting
isModified	false

You will receive a list of meetings:

The screenshot shows the API response for the 'List Meetings' endpoint. The response is a JSON object with a status of 200. The 'Data' tab is active, showing a list of meetings. The first meeting in the list is detailed below:

```
{
  "items": [
    {
      "id": "11556412d0114b1ab6e95f4de6b22f4b_I_678181572335116526",
      "meetingSeriesId": "11556412d0114b1ab6e95f4de6b22f4b_I_678181572335116526",
      "scheduledMeetingId": "11556412d0114b1ab6e95f4de6b22f4b_I_678181572335116526",
      "title": "Charles Holland's meeting",
      "agenda": "",
      "meetingType": "meeting",
      "state": "ended",
      "timezone": "UTC",
      "start": "2025-09-28T20:23:06Z",
      "end": "2025-09-28T20:24:01Z",
      "hostUserId": "Y2lzY29zcGFyZ3VzL1BFT1BMR",
      "hostDisplayName": "Charles Holland",
      "hostEmail": "The value is hidden by portal",
      "siteUrl": "cb12702.webex.com",
      "webLink": "The value is hidden by portal",
      "scheduledType": "meeting",
      "hasChat": false,
      "hasRecording": false,
      "hasTranscription": false,
      "hasClosedCaption": false
    }
  ]
}
```

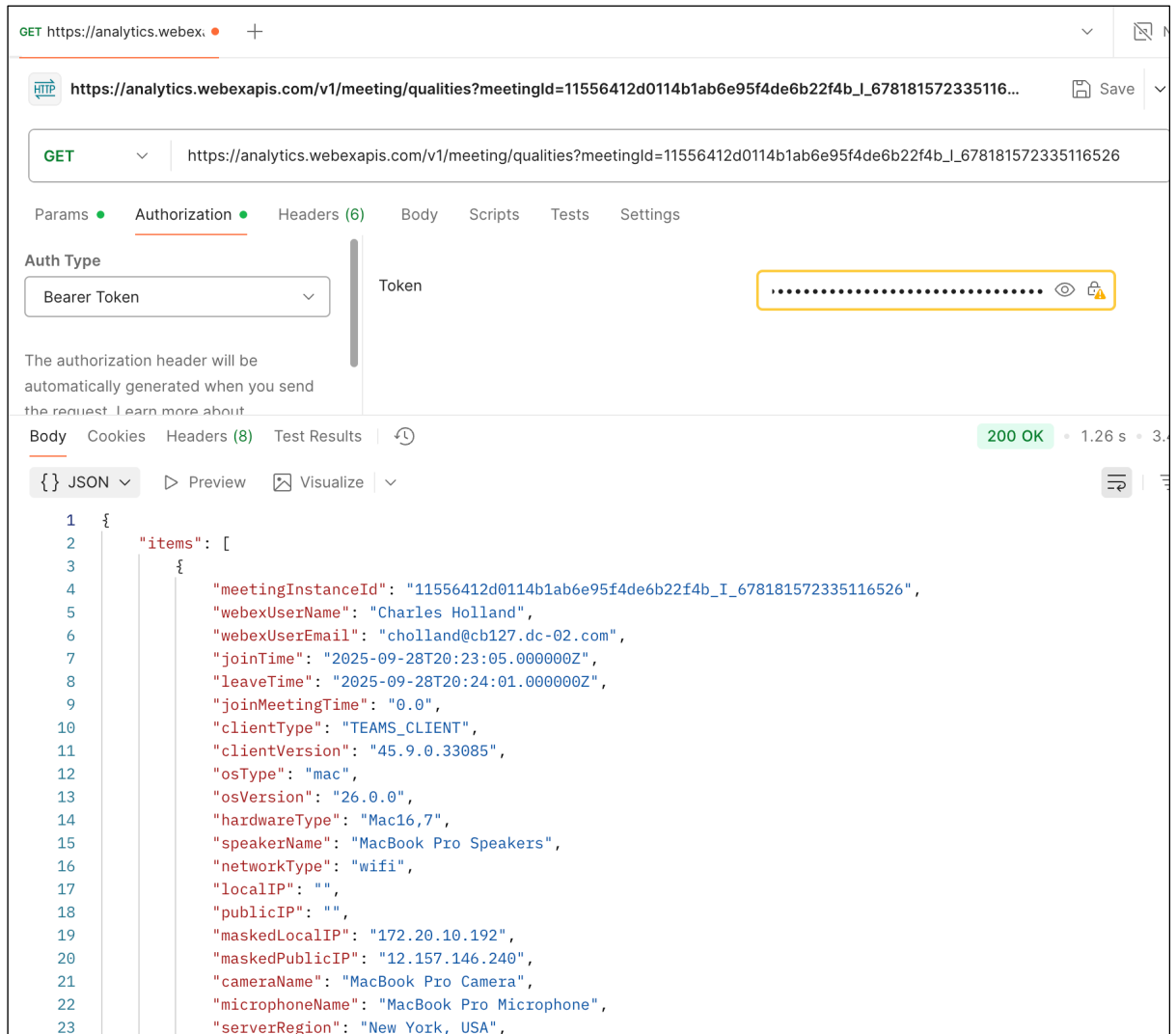
Pick one and ensure the ID follows a format similar to:

- 11556412d0114b1ab6e95f4de6b22f4b_I_678181572335116526

2. Get Meeting Qualities:

Retrieve quality metrics for a specific meeting using this endpoint:

<https://analytics.webexapis.com/v1/meeting/qualities?meetingId=<meetingId>>



The response from the Meetings API includes detailed information for each meeting participant:

- **Identification:** Meeting and participant IDs, user name, and email
- **Timing:** Join and leave times, and time taken to join the meeting
- **Client and Device:** Client type and version, OS type and version, hardware details, camera, microphone, and speakers used
- **Network Info:** Network type, masked local and public IP addresses, server region
- **Media Quality Metrics:** Audio, video, and content sharing statistics (packet loss, latency, jitter, frame rate, resolution, codec, bitrate, and transport type for both inbound and outbound streams)
- **Resource Usage:** Average and maximum CPU usage during the meeting

These details help you analyze participant experience, troubleshoot meeting quality issues, and identify patterns related to devices, networks, or client versions.